

A Dual-Layer ECC Methodology for Client-Server Authentication and Server-Server Data Protection in Electronic Finance Systems

PhD Dissertation in Information Security

Submitted in partial fulfillment of the requirements for the degree of Doctor
of Philosophy

September 14, 2026

Abstract

This dissertation investigates information security challenges in electronic banking and electronic finance service systems, with particular emphasis on authentication, key exchange, data encryption, and integrity protection. A comprehensive analysis of contemporary attack mechanisms—including phishing, man-in-the-middle attacks, replay attacks, session hijacking, credential stuffing, and insider threats—establishes the threat landscape and identifies critical limitations in existing security mechanisms. Two novel elliptic curve cryptography (ECC) based schemes are proposed: ECC-DTB-AKA (Device- and Transaction-Bound Authenticated Key Agreement) for client-server channels, and ECC-HISE (Hierarchical Integrity-preserving Secure Envelope) for server-server batch data transfer. Both schemes are formally analyzed, quantitatively evaluated against established baselines, and implemented in a working software prototype. Experimental results demonstrate that ECC-DTB-AKA achieves mutual authentication in three communication rounds with device and transaction binding, while ECC-HISE provides hierarchical key management with Merkle-tree audit chains enabling efficient batch integrity verification.

Contents

Abstract	iii
Preface	1
0.1 Background and Motivation	1
0.2 Development and Future Prospects of E-Banking and E-Finance .	2
0.3 The Importance of Security Systems in E-Banking and E-Finance	4
0.4 Research Objectives and Scope	5
0.5 Summary of Chapters	6
0.6 Acknowledgments	7
0.7 Methodological Approach	8
0.8 Structure of the E-Financial Service System	8
0.9 E-Banking in the Global Context	9
0.10 Notation and Conventions	10
1 Threat Landscape and Security Gaps	13
1.1 Attack Mechanisms in E-Banking and E-Finance Services	13
1.1.1 Taxonomy of Attacks	14
1.1.2 Case Studies	15
1.1.3 Attack Trends and Statistics	16
1.1.4 Regulatory and Standards Landscape	17
1.1.5 Detailed Analysis of Selected Attack Vectors	18
1.1.6 Impact Assessment Framework	19
1.2 Authentication and Key Exchange in E-Banking Systems	20
1.2.1 Client-Server Authentication Mechanisms	20
1.2.2 Server-Server Authentication and Key Exchange	22

1.2.3	Identified Limitations and Drawbacks	22
1.2.4	Elliptic Curve Cryptography in Financial Authentication	24
1.2.5	Comparative Overview of Authentication Protocols	25
1.2.6	Password-Based Authentication Vulnerabilities	26
1.3	Data Encryption, Security, and Integrity in E-Banking	27
1.3.1	Data Encryption Mechanisms	27
1.3.2	Data Integrity and Audit Mechanisms	27
1.3.3	Identified Limitations and Drawbacks	28
1.3.4	Key Management in Financial Institutions	29
1.3.5	Merkle Trees and Cryptographic Audit Trails	30
1.3.6	Homomorphic Encryption and Emerging Approaches	31
1.4	Threat Model, Assumptions, and Research Questions	32
1.4.1	Threat Model	32
1.4.2	System Assumptions	32
1.4.3	Research Questions	33
1.4.4	Chapter Summary	34
1.4.5	Security Requirements Derivation	34
1.4.6	Relationship to Subsequent Chapters	35
2	ECC-DTB-AKA Authenticated Key Agreement	37
2.1	Introduction and Motivation	37
2.1.1	Limitations of Layered Security Architectures	38
2.2	Design Goals and Requirements	39
2.2.1	Elliptic Curve Cryptography Foundations	39
2.3	Protocol Specification	40
2.3.1	Notation and Prerequisites	40
2.3.2	Round 1 — Client Init (MSG1)	40
2.3.3	Round 2 — Server Response (MSG2)	41
2.3.4	Round 3 — Client Confirm (MSG3)	42
2.3.5	Transaction Key Derivation	42
2.3.6	Message Format Summary	43
2.3.7	Algorithm: Client Handshake	43
2.3.8	Algorithm: Server Handshake	44
2.3.9	Error Handling and Abort Conditions	44
2.4	Security Analysis	45

2.4.1	Security Model and Assumptions	45
2.4.2	Security Properties and Proof Sketches	45
2.4.3	Attack Resistance Analysis	46
2.4.4	Session Lifecycle and Key Erasure	46
2.4.5	Reduction to Standard Assumptions	47
2.4.6	Comparison with Bellare-Rogaway KEA Protocol	47
2.5	Effectiveness Analysis and Quantitative Evaluation	48
2.5.1	Experimental Setup	48
2.5.2	Authentication Latency	48
2.5.3	Message Size and Communication Overhead	49
2.5.4	Communication Steps and Scalability	50
2.5.5	Security-Performance Trade-off Summary	50
2.5.6	Resistance to Limitations from Section 1.2.3	50
2.5.7	Computational Cost Breakdown	51
2.5.8	Network Impact Analysis	51
2.6	Comparison with Prior Authentication Approaches	52
2.6.1	TLS-Centric Architectures	53
2.6.2	OAuth 2.0 and Open Banking	53
2.6.3	FIDO2 and Phishing-Resistant Authentication	53
2.7	Chapter Summary	54
3	ECC-HISE Secure Envelope	55
3.1	Introduction and Motivation	55
3.2	Design Goals and Requirements	56
3.3	Hierarchical Key Management	56
3.3.1	Master, Domain, and Session Keys	56
3.3.2	Key Rotation and Revocation	57
3.3.3	Comparison with Flat Key Management	57
3.4	Secure Envelope Construction	58
3.4.1	Record Envelope Algorithm	58
3.4.2	Batch Header and Merkle Root	58
3.4.3	Verification and Decryption	59
3.4.4	Deployment in Inter-Bank Settlement	59
3.5	Security Analysis	60
3.5.1	Security Properties	60

3.5.2	Attack Resistance	61
3.5.3	Addressing Section 1.3.3 Limitations	61
3.6	Effectiveness Analysis and Quantitative Evaluation	61
3.6.1	Experimental Setup	62
3.6.2	Throughput and Latency	62
3.6.3	Storage and Bandwidth Overhead	62
3.6.4	Audit Efficiency	63
3.6.5	Scalability Projection	63
3.7	Comparison with Chapter 2 and Prior Approaches	64
3.8	Chapter Summary	64
4	Architecture and Implementation	65
4.1	Security System Architecture of an E-Financial Service System . .	65
4.1.1	Architectural Layers and Trust Boundaries	65
4.1.2	Component Interaction Flow	66
4.1.3	Technology Stack	67
4.2	Programmatic Implementation of ECC-DTB-AKA (Chapter 2) . .	67
4.2.1	Module Structure	68
4.2.2	Core Classes and Data Structures	68
4.2.3	REST API Integration	69
4.2.4	Test Results	69
4.2.5	Key Derivation Implementation Details	69
4.3	Programmatic Implementation of ECC-HISE (Chapter 3)	70
4.3.1	Module Structure	70
4.3.2	Envelope Construction and Verification	70
4.3.3	Settlement API Integration	71
4.3.4	Test Results and Performance	71
4.3.5	Merkle Tree Implementation	71
4.4	Integrated System Demonstration	72
4.4.1	Deployment Considerations	72
4.4.2	Limitations of the Prototype	72
4.5	Chapter Summary	73

5	Evaluation and Prior Work	75
5.1	Effectiveness of Proposed Methods Against Network Attacks . . .	75
5.1.1	Attack Resistance Matrix	75
5.1.2	Quantitative Authentication Performance	75
5.1.3	Case-Based Effectiveness Assessment	77
5.2	Prior Work on Authentication and Key Exchange	77
5.2.1	TLS 1.3 Handshake Protocol	78
5.2.2	OAuth 2.0 and Open Banking (PKCE)	78
5.2.3	FIDO2 / WebAuthn	79
5.2.4	Server-Server Key Exchange (mTLS and SWIFT BKE) . .	79
5.2.5	Comparative Summary	80
5.2.6	EMV and Card-Present Authentication	80
5.3	Prior Work on Data Encryption, Security, and Integrity	81
5.3.1	TLS Record Layer Protection	81
5.3.2	ECIES and Hybrid Encryption	81
5.3.3	Static Pre-Shared Key File Encryption	82
5.3.4	Blockchain and Distributed Ledger Audit	82
5.3.5	Comparative Summary	82
5.3.6	Database TDE and Column-Level Encryption	83
5.4	Synthesis: Vulnerabilities and Research Contributions	83
5.4.1	Vulnerabilities in Existing Systems	83
5.4.2	Research Contributions	84
5.4.3	Limitations and Future Work	85
5.4.4	Alignment with Regulatory Requirements	85
5.5	Chapter Summary	85
	Conclusions	87
5.6	Summary of Approach	87
5.7	Contributions	87
5.8	Limitations	88
5.9	Future Work	88
A	Appendix	89
A.1	A. Algorithm Listings	89
A.2	B. API Specification	89

A.3	C. Benchmark Raw Data	89
A.4	D. Prototype Module Index	90
B	Scheme Design Document	91
B.1	1. Overview	91
B.2	2. Scheme A: ECC-DTB-AKA	91
	B.2.1 Notation	92
	B.2.2 Protocol Steps (3-Round Handshake)	92
	B.2.3 Security Properties	92
B.3	3. Scheme B: ECC-HISE	93
	B.3.1 Key Hierarchy	93
	B.3.2 Envelope Construction	93
	B.3.3 Verification and Audit	93
B.4	4. Comparison Baselines	94
B.5	5. Implementation Mapping	94
	References	95

List of Figures

1.1	Attack lifecycle flowchart: reconnaissance <<< <i>PH0</i> >>> initial access <<< <i>PH1</i> >>> lateral movement <<< <i>PH2</i> >>> privilege escalation <<< <i>PH3</i> >>> data exfiltration/fraud <<< <i>PH4</i> >>> cover tracks (Figure 1.1).	16
1.2	Current client-server authentication flow: TLS handshake <<< <i>PH0</i> >>> credential submission <<< <i>PH1</i> >>> MFA challenge <<< <i>PH2</i> >>> session token issuance (Figure 1.2).	21
1.3	Server-server authentication in a microservice e-finance architecture with mTLS service mesh (Figure 1.3).	23
1.4	Data protection layers in e-banking: at-rest (TDE), in-transit (TLS), in-use (confidential computing) (Figure 1.4).	28
1.5	Threat model diagram showing adversary position relative to client, bank server, and inter-bank network (Figure 1.5).	33
2.1	ECC-DTB-AKA three-round handshake sequence diagram (Figure 2.1).	41
2.2	Key derivation hierarchy from ECDH shared secret to MSK and TBK_{txn} (Figure 2.2).	42
2.3	Quantitative comparison of authentication latency and message size (Figure 2.3).	49
3.1	ECC-HISE three-level key hierarchy: Master Key, Domain Keys, Session Keys (Figure 3.1).	57
3.2	ECC-HISE secure envelope structure with Merkle audit chain (Figure 3.2).	59
3.3	Server-server batch verification and audit flow (Figure 3.3).	60

3.4	Batch processing latency and size comparison across schemes (Figure 3.4).	62
4.1	Integrated e-financial service system architecture (Figure 4.1).	66
4.2	End-to-end transaction and settlement flow (Figure 4.2).	67
4.3	ECC-DTB-AKA software module diagram (Figure 4.3).	68
4.4	ECC-HISE software module diagram (Figure 4.4).	70
4.5	Deployment topology for prototype services (Figure 4.5).	72
5.1	Comparative attack resistance overview (Figure 5.1).	76
5.2	Combined quantitative comparison of both schemes (Figure 5.2).	77
5.3	TLS 1.3 full handshake flow (Figure 5.3).	78
5.4	OAuth 2.0 PKCE authorization flow for open banking (Figure 5.4).	79
5.5	mTLS service mesh authentication in banking microservices (Figure 5.5).	80
5.6	TLS record layer vs. persistent envelope integrity (Figure 5.6).	81
5.7	Merkle audit: blockchain vs. ECC-HISE envelope (Figure 5.7).	82

List of Tables

1.1	Table 1.1: Taxonomy of attack mechanisms against e-banking systems.	15
1.2	Table 1.2: Limitations of current authentication and key-exchange mechanisms.	24
1.3	Table 1.4: Comparative overview of authentication protocols for e-banking.	26
1.4	Table 1.3: Limitations of current data security mechanisms and proposed directions.	30
2.1	Table 2.3: ECC-DTB-AKA message field specification.	43
2.2	Table 2.1: ECC-DTB-AKA resistance to e-banking attack mechanisms.	46
2.3	Table 2.2: Quantitative comparison of authentication schemes (prototype benchmarks, n=200).	50
2.4	Table 2.4: Projected end-to-end authentication time for ECC-DTB-AKA.	52
3.1	Table 3.1: ECC-HISE resistance to server-server data attacks. . . .	61
3.2	Table 3.2: Quantitative comparison for 100-record settlement batches (n=50 iterations).	63
4.1	Table 4.1: Architecture components and technology mapping. . .	67
4.2	Table 4.2: ECC-DTB-AKA prototype test results.	69
4.3	Table 4.3: ECC-HISE prototype test results.	71

5.1	Table 5.1: Attack resistance matrix for proposed schemes vs. base- lines.	76
5.2	Table 5.2: Authentication and key-exchange prior work vs. ECC- DTB-AKA.	80
5.3	Table 5.3: Data encryption and integrity prior work vs. ECC-HISE.	83
5.4	Table 5.4: Vulnerabilities in existing e-banking systems and disser- tation responses.	84

Preface

0.1 Background and Motivation

Electronic banking and electronic finance service systems have undergone a profound transformation over the past three decades. What began in the 1980s as rudimentary home-banking terminals connected via dial-up modems has evolved into a globally interconnected ecosystem comprising mobile banking applications, real-time payment rails, open-banking APIs, cryptocurrency custody services, and algorithmic trading platforms. According to industry reports, more than three billion individuals worldwide now access financial services through digital channels, and the volume of electronic payment transactions exceeds one trillion events annually. This digitization has delivered unprecedented convenience, financial inclusion, and operational efficiency, yet it has simultaneously expanded the attack surface available to adversaries ranging from opportunistic cybercriminals to sophisticated nation-state actors [42][43].

The security of e-banking and e-finance systems is not merely a technical concern; it is a foundational requirement for economic stability and public trust. A single successful breach can result in direct financial losses measured in hundreds of millions of dollars, regulatory penalties, reputational damage that persists for years, and erosion of confidence in the broader financial system. High-profile incidents—including the 2016 Bangladesh Bank heist involving fraudulent SWIFT messages, the 2017 Equifax data breach affecting 147 million consumers, and numerous credential-stuffing campaigns targeting retail banking portals—demonstrate that existing security architectures, despite decades of investment, remain vulnerable to both novel and well-known attack vectors [24][20][21][22].

Cryptography serves as the bedrock upon which e-banking security is built.

From the SSL/TLS protocols that protect data in transit, to the EMV chip-and-PIN standards governing card present transactions, to the hardware security modules (HSMs) safeguarding root keys in data centers, cryptographic mechanisms are deeply embedded in every layer of the financial technology stack. Among the cryptographic primitives available today, elliptic curve cryptography (ECC) has emerged as the dominant choice for new deployments due to its favorable security-to-key-size ratio: a 256-bit elliptic curve key provides security comparable to a 3072-bit RSA key, resulting in lower computational overhead, reduced bandwidth consumption, and improved performance on resource-constrained devices such as smartphones and IoT terminals [11][34][35][16].

This dissertation addresses a critical gap at the intersection of ECC-based authentication, key exchange, and data protection in e-banking environments. While substantial prior research has investigated individual aspects of financial system security—ranging from multi-factor authentication protocols to blockchain-based audit trails—few works have proposed integrated methodologies that simultaneously strengthen client-server authentication channels and server-server data-transfer channels using coherent ECC-based designs, accompanied by rigorous analysis, quantitative evaluation, and practical implementation. The research presented herein aims to fill this gap by proposing, analyzing, implementing, and evaluating two novel schemes: ECC-DTB-AKA (Device- and Transaction-Bound Authenticated Key Agreement) and ECC-HISE (Hierarchical Integrity-preserving Secure Envelope) [7][8][29][31].

0.2 Development and Future Prospects of E-Banking and E-Finance

The historical trajectory of electronic banking can be divided into four distinct generations. The first generation (1980s–1990s) featured proprietary dial-up systems with limited functionality, primarily balance inquiry and fund transfers between accounts at the same institution. The second generation (late 1990s–2000s) coincided with the commercialization of the World Wide Web, giving rise to internet banking portals protected by SSL/TLS and username-password authentication. The third generation (2010s) was driven by smartphone adoption, introducing mobile banking applications, biometric authentication, push notifications for transaction verification,

and near-field communication (NFC) payments. We are now entering the fourth generation, characterized by open banking regulations (such as the European PSD2 directive and similar frameworks in the United Kingdom, Australia, and Brazil), real-time gross settlement (RTGS) systems operating 24/7, central bank digital currencies (CBDCs), and decentralized finance (DeFi) protocols that challenge traditional intermediation models [18][38][1][28].

Several technological trends will shape the future of e-finance services and, by extension, the security requirements that must be satisfied. First, the proliferation of application programming interfaces (APIs) under open-banking mandates creates new client-server and server-server communication patterns in which third-party providers access customer account data and initiate payments on behalf of users. Each API endpoint represents a potential entry point for attackers, necessitating robust authentication and fine-grained authorization mechanisms. Second, the migration of banking workloads to cloud infrastructure introduces shared-responsibility security models in which traditional network perimeters dissolve and identity becomes the primary security boundary. Third, the increasing use of artificial intelligence and machine learning in fraud detection, credit scoring, and customer service creates new categories of adversarial attacks, including model poisoning and inference attacks on sensitive training data.

Quantum computing poses a longer-term but strategically significant threat to current cryptographic infrastructure. Although cryptographically relevant quantum computers have not yet materialized, financial institutions with multi-decade data confidentiality requirements must begin planning for post-quantum migration. The U.S. National Institute of Standards and Technology (NIST) has standardized post-quantum algorithms (ML-KEM, ML-DSA, SLH-DSA), and regulatory bodies are expected to issue timelines for transition. The ECC-based schemes proposed in this dissertation are designed with algorithm agility in mind, enabling future replacement of specific primitives without altering the overall protocol architecture [39].

Looking ahead, e-finance service systems will likely converge toward a hybrid architecture combining centralized trust anchors (banks, clearing houses, regulators) with distributed ledger technologies for settlement and audit. Security systems must evolve to protect not only traditional account-based transactions but also tokenized assets, smart-contract interactions, and cross-border payment corridors operating

across heterogeneous regulatory jurisdictions. The methodologies developed in this thesis provide a foundation that can be extended to these emerging paradigms.

0.3 The Importance of Security Systems in E-Banking and E-Finance

Information security in electronic finance is commonly analyzed through the CIA triad (Confidentiality, Integrity, Availability) supplemented by authenticity, non-repudiation, and accountability. Each property carries particular significance in the banking domain.

Confidentiality ensures that sensitive financial data—including account balances, transaction histories, personal identification information, and cryptographic keys—remains accessible only to authorized parties. Breaches of confidentiality can facilitate identity theft, targeted phishing, and insider trading. In e-banking systems, confidentiality must be maintained across multiple states: data at rest (stored in databases and backup systems), data in transit (moving across networks between clients, servers, and partner institutions), and data in use (being processed in memory during transaction authorization) [42][43][22].

Integrity guarantees that financial records and transaction messages have not been altered without detection. The integrity requirement is especially stringent in inter-bank settlement, where a modification to a single field in a payment instruction can redirect millions of dollars to an attacker-controlled account. Cryptographic mechanisms such as message authentication codes (MACs), digital signatures, and hash chains provide integrity protection, but their effectiveness depends on correct key management and comprehensive coverage of all data paths [3][26][9].

Availability ensures that banking services remain accessible to legitimate users when needed. Distributed denial-of-service (DDoS) attacks against banking infrastructure have become increasingly common, with some attacks exceeding one terabit per second in traffic volume. While availability is often treated separately from cryptographic security, the two concerns intersect when authentication protocols are designed to resist resource-exhaustion attacks and when rate-limiting mechanisms must not inadvertently lock out legitimate customers.

Authenticity and non-repudiation are particularly critical in financial contexts where disputes over transaction authorization arise frequently. A customer may

claim that a wire transfer was unauthorized, while the bank asserts that valid credentials were presented. Strong mutual authentication protocols and digital signatures with long-term verifiability provide evidentiary support in such disputes. Regulatory frameworks including the Payment Card Industry Data Security Standard (PCI DSS), the Sarbanes-Oxley Act, and the General Data Protection Regulation (GDPR) impose legal obligations that translate directly into technical security requirements [17][26][9].

The defense-in-depth principle dictates that no single security control should constitute a single point of failure. E-banking systems typically deploy firewalls, intrusion detection systems, security information and event management (SIEM) platforms, multi-factor authentication, encryption, and physical security for data centers in layered combination. However, the cryptographic layer—the focus of this dissertation—remains the ultimate guarantor of confidentiality and integrity when all other layers have been bypassed [42][43].

0.4 Research Objectives and Scope

The primary objective of this research is to design, analyze, implement, and evaluate novel ECC-based security schemes that address documented vulnerabilities in contemporary e-banking and e-finance systems. Specifically, the research pursues the following goals: [42][43]

First, to systematically catalog and analyze attack mechanisms currently deployed against e-banking and e-finance services, establishing a threat landscape that motivates the proposed countermeasures (Chapter 1). Second, to propose ECC-DTB-AKA, a device- and transaction-bound authenticated key agreement protocol for client-server channels, and to demonstrate its superiority over existing approaches in terms of security properties, communication efficiency, and computational overhead (Chapter 2). Third, to propose ECC-HISE, a hierarchical integrity-preserving secure envelope scheme for server-server data transfer, addressing limitations in current encryption and audit mechanisms (Chapter 3). Fourth, to implement both schemes in a working software prototype and to describe the security system architecture of an e-financial service system that integrates them (Chapter 4). Fifth, to conduct a comprehensive evaluation including review of prior work, quantitative performance comparison, and analysis of resistance to the attacks

identified in Chapter 1 (Chapter 5) [7][8][42][43].

The scope of this dissertation is deliberately focused on the cryptographic and protocol layers of e-banking security. Physical security, social engineering, organizational policy, and regulatory compliance are acknowledged as important but are not the primary subject of investigation. The implementation prototype uses software-based key storage rather than hardware security modules, and the formal security proofs are presented as structured proof sketches rather than machine-verified derivations. These limitations are discussed frankly in the conclusions [42][43].

0.5 Summary of Chapters

Chapter 1 establishes the foundation of this dissertation by defining and categorizing attack mechanisms currently used against e-banking and e-finance services, including phishing, man-in-the-middle attacks, replay attacks, session hijacking, credential stuffing, and insider threats. The chapter presents real-world case studies illustrating the impact of successful attacks, reviews the current state of authentication and key-exchange mechanisms (Section 1.2) and data encryption and integrity mechanisms (Section 1.3), and identifies their respective limitations. Section 1.4 formalizes the threat model, system assumptions, and research questions that guide the subsequent chapters [42][43][22].

Chapter 2 presents the first proposed methodology: ECC-DTB-AKA. This chapter describes an alternative approach to strengthening client-server security in e-banking systems through a novel authenticated key agreement protocol that binds session keys to device fingerprints and transaction nonces. The chapter includes detailed algorithmic descriptions, protocol flowcharts, a formal security analysis, and quantitative evaluation comparing ECC-DTB-AKA against TLS 1.3, standard ECDH, and OAuth 2.0 PKCE in terms of authentication latency, message overhead, and communication round trips [1][7][8][12].

Chapter 3 presents the second proposed methodology: ECC-HISE. Taking a fundamentally different approach from Chapter 2, this chapter addresses server-server data protection through a hierarchical key management scheme combined with authenticated encryption and Merkle-tree audit chains. The scheme overcomes drawbacks of prior data-security approaches identified in Section 1.3, and the chap-

ter provides algorithmic specifications, architectural diagrams, and effectiveness analysis [36][2][3][30].

Chapter 4 describes the practical application of both proposed schemes. Section 4.1 presents the security system architecture of an e-financial service system integrating ECC-DTB-AKA and ECC-HISE. Sections 4.2 and 4.3 detail the programmatic implementation of the Chapter 2 and Chapter 3 schemes, respectively, including module structure, API design, and deployment considerations. Block diagrams and flowcharts visually represent the integrated system.

Chapter 5 provides comprehensive analysis and evaluation. Section 5.1 assesses the effectiveness of the proposed methods against the attacks cataloged in Chapter 1. Section 5.2 reviews prior work on authentication and key exchange in server-server and client-server environments. Section 5.3 reviews prior work on data encryption, data security, and integrity in e-banking. Section 5.4 synthesizes findings from Sections 5.1 through 5.3, lists security vulnerabilities present in existing systems, and identifies the issues addressed by this dissertation [8][29][42][43].

The concluding remarks summarize the contributions of this research, acknowledge limitations, and suggest directions for future work. The appendix provides complete algorithm listings, test vectors, benchmark raw data, and API specifications to support reproducibility.

0.6 Acknowledgments

The completion of this dissertation would not have been possible without the support and guidance of numerous individuals and institutions. I express my sincere gratitude to my doctoral supervisor for invaluable advice throughout the research process. I thank the faculty and staff of the Department of Information Security for providing an environment conducive to rigorous scholarship. I am also grateful to the developers of open-source cryptographic libraries whose tools enabled the implementation work described in Chapter 4, and to the authors of the foundational papers reviewed in Chapter 5, whose contributions form the intellectual context within which this research is situated.

0.7 Methodological Approach

The research methodology employed in this dissertation follows a design-science research paradigm, which is well suited to information security investigations that aim to create and evaluate novel artifacts (in this case, cryptographic protocols and their software implementations) to solve identified organizational and technical problems. The methodology comprises five iterative phases: problem identification, solution design, prototype development, demonstration, and evaluation.

In the problem identification phase (Chapter 1), we systematically catalog attack mechanisms and analyze limitations of existing authentication, key-exchange, and data-protection mechanisms in e-banking systems. The solution design phase (Chapters 2 and 3) produces formal protocol specifications for ECC-DTB-AKA and ECC-HISE, accompanied by security analysis and comparative evaluation against established baselines. The prototype development phase (Chapter 4) translates protocol specifications into working software modules using Python and the OpenSSL-backed cryptography library. The demonstration phase exercises the prototype through representative e-banking scenarios including customer login, fund transfer, and inter-bank settlement batch processing. The evaluation phase (Chapter 5) measures quantitative performance metrics and assesses qualitative security properties against the threat model defined in Section 1.4 [42][43].

Elliptic curve cryptography is selected as the foundational primitive for both proposed schemes based on three criteria: (1) security efficiency — ECC P-256 provides approximately 128 bits of security with a 256-bit key, compared to 3072-bit RSA keys for equivalent strength; (2) industry adoption — ECC is mandated or recommended by NIST, EMV, FIDO2, and TLS 1.3 specifications; and (3) implementation maturity — production-grade open-source libraries (OpenSSL, lib-sodium, Python cryptography) provide audited implementations of ECDH, ECDSA, and Ed25519 on standard curves [1][9][26][10].

0.8 Structure of the E-Financial Service System

Throughout this dissertation, we reference a conceptual e-financial service system that serves as the deployment context for the proposed security schemes. This system comprises four logical tiers: the client tier, the application tier, the settlement

tier, and the governance tier.

The client tier includes mobile banking applications, web browsers, and ATM terminals that customers use to access financial services. Clients communicate with the application tier over public networks (the Internet) and must authenticate themselves and establish encrypted sessions before performing transactions. The ECC-DTB-AKA protocol operates primarily in this tier, securing the client-server channel.

The application tier consists of API gateways, authentication services, account management services, and transaction processing engines. These components are deployed as containerized microservices behind load balancers, with mutual TLS providing transport security between services. Application-tier services validate customer requests, enforce business rules, and initiate settlement operations [1][28].

The settlement tier handles inter-bank and intra-bank batch processing, including end-of-day reconciliation, regulatory reporting, and payment clearing. Data transferred between settlement servers at different institutions requires confidentiality, integrity, and auditability beyond what transport-layer security alone provides. The ECC-HISE scheme operates in this tier, protecting batch data with hierarchical keys and Merkle audit chains [30].

The governance tier encompasses certificate authorities, hardware security modules, security operations centers, and regulatory reporting interfaces. The clearing-house authority manages the master key for ECC-HISE's hierarchical key derivation, while the bank's internal CA issues certificates used in ECC-DTB-AKA server authentication [4][5][6].

0.9 E-Banking in the Global Context

The global e-banking landscape exhibits significant regional variation in technology adoption, regulatory frameworks, and security postures [42][43]. In Scandinavia, nearly 100% of banking customers use digital channels, with mobile payment systems such as Swish and MobilePay achieving near-universal adoption. Strong government digital identity infrastructure (BankID in Sweden and Norway, NemID/MitID in Denmark) provides a foundation for cryptographic authentication that exceeds the security of username-password systems common in other regions.

In East Asia, super-app ecosystems integrate banking, payments, social media,

and e-commerce into single platforms (WeChat Pay, Alipay, LINE Pay). These platforms process transaction volumes exceeding those of entire countries' GDP, creating unique scalability and security challenges. QR-code-based payment authentication, while convenient, has been targeted by attackers who replace legitimate merchant QR codes with fraudulent ones—a threat vector that requires binding payment authorization to verified merchant identity.

In Africa, mobile money services (M-Pesa, MTN Mobile Money) have brought banking to underbanked populations through feature phones and SMS-based interfaces. The security models for these systems often rely on SIM-card identity and PIN codes, making them vulnerable to SIM-swapping attacks that have resulted in significant financial losses. The ECC-DTB-AKA scheme's device-binding property is particularly relevant in markets where SIM-based identity is the primary authentication factor.

In North America and Western Europe, open-banking initiatives are reshaping the competitive landscape by requiring banks to expose APIs to licensed third-party providers. This architectural shift multiplies the number of authentication and authorization endpoints, each requiring robust protection. The server-server security requirements for inter-institutional data exchange—addressed by ECC-HISE—become increasingly critical as payment initiation flows traverse multiple service providers before reaching the customer's bank.

0.10 Notation and Conventions

Throughout this dissertation, the following notation and typographic conventions are employed unless otherwise stated. Scalar values and binary strings are denoted in lowercase italic (e.g., k , nonce). Group elements on elliptic curves are denoted in uppercase italic (e.g., P , Q , G). Functions and algorithms are denoted in upright font (e.g., H , HKDF , ECDH). Sets are denoted in calligraphic or blackboard-bold font (e.g., $\mathbb{Z}_q$ for integers modulo q) [5][6][11][34].

The symbol $\parallel$ denotes concatenation of bit strings. The symbol $\lll PH0 \ggg$ denotes uniform random sampling from a set. The symbol $\lll PH1 \ggg$ denotes deterministic output of a function. $A \lll PH2 \ggg_P B$ indicates that entity A sends message to entity B over protocol P . Security parameters are expressed in bits (e.g., 128-bit security, 256-bit key).

Chapter and section references use decimal notation (e.g., Section 1.2.3). Figure and table references use chapter-qualified numbering (e.g., Figure 1.2, Table 1.3). Algorithm pseudocode is presented in numbered blocks in the appendix and referenced from the main text. All cryptographic algorithms use well-established standards: SHA-256 per FIPS 180-4, HKDF per RFC 5869, AES-256-GCM per NIST SP 800-38D, ECDSA per FIPS 186-5, and Ed25519 per RFC 8032 [\[2\]\[5\]\[6\]\[9\]](#).

Chapter 1

Threat Landscape and Security Gaps

Attack Mechanisms, Threat Landscape, and Security Gaps in E-Banking Systems

1.1 Attack Mechanisms in E-Banking and E-Finance Services

Electronic banking and electronic finance service systems operate in one of the most adversarial computing environments in existence. The concentration of monetary value, the interconnectivity of financial networks, and the diversity of access channels create opportunities for attackers at every layer of the technology stack. This section provides a systematic definition and taxonomy of attack mechanisms currently employed against e-banking and e-finance services, supported by representative case studies drawn from documented security incidents [42][43].

An attack mechanism, in the context of this dissertation, is defined as a systematic method or technique employed by an adversary to exploit a vulnerability in an e-banking or e-finance system component—whether technical, procedural, or human—in order to achieve an unauthorized objective such as theft of funds, exfiltration of sensitive data, disruption of service, or compromise of system integrity. Attack mechanisms differ from individual exploits in that they describe reusable patterns that can be adapted to multiple targets and environments [42][43].

1.1.1 Taxonomy of Attacks

We classify attack mechanisms against e-banking systems into six primary categories based on the attacker's objective and the layer of the system stack targeted: (1) credential and authentication attacks, (2) communication channel attacks, (3) application-layer attacks, (4) infrastructure attacks, (5) insider and supply-chain attacks, and (6) social engineering attacks. Table 1.1 summarizes these categories with representative techniques [42][43].

Credential and authentication attacks target the mechanisms by which users and systems prove their identity. Techniques include credential stuffing (automated injection of username-password pairs leaked from unrelated breaches), brute-force password guessing, pass-the-hash attacks against Windows authentication in bank internal networks, and exploitation of weak multi-factor authentication implementations that rely on SMS one-time passwords (OTPs) vulnerable to SIM-swapping.

Communication channel attacks intercept, modify, or replay data in transit between clients and servers or between server nodes. The classic man-in-the-middle (MITM) attack positions the adversary between communicating parties, potentially decrypting, altering, and re-encrypting messages. SSL stripping downgrades HTTPS connections to HTTP, while BGP hijacking reroutes network traffic through attacker-controlled routers. Replay attacks capture valid authentication messages and retransmit them to gain unauthorized access.

Application-layer attacks exploit vulnerabilities in banking software itself. SQL injection manipulates database queries through unsanitized user input. Cross-site scripting (XSS) injects malicious scripts into banking web pages viewed by other users. Cross-site request forgery (CSRF) tricks authenticated users into submitting unintended transactions. Business logic flaws may allow negative-amount transfers or bypass of transaction limits.

Infrastructure attacks target the servers, networks, and cloud platforms hosting banking services. Distributed denial-of-service (DDoS) attacks overwhelm servers with traffic, rendering services unavailable. Ransomware encrypts critical banking data and demands payment for decryption keys. Container escape attacks in cloud-hosted banking microservices can compromise co-located workloads.

Insider and supply-chain attacks leverage trusted positions or compromised third-party components. Malicious insiders with database access can exfiltrate customer records or manipulate account balances. Supply-chain compromises, such

as the 2020 SolarWinds incident, insert backdoors into software updates distributed to thousands of organizations including financial institutions.

Social engineering attacks manipulate human psychology rather than technical vulnerabilities. Phishing emails impersonate banks to harvest credentials. Vishing (voice phishing) uses phone calls to trick customers into revealing authentication codes. Pretexting creates fabricated scenarios to persuade bank employees to override security controls [22].

Category	Techniques	Primary Target	Impact
Credential/Auth	Stuffing, brute force, SIM-swap	Login systems	Account takeover
Channel	MITM, SSL strip, replay, BGP hijack	Network layer	Data theft, fraud
Application	SQLi, XSS, CSRF, logic flaws	Banking apps	Data breach, fraud
Infrastructure	DDoS, ransomware, container escape	Servers/cloud	Outage, data loss
Insider/Supply-chain	Data exfiltration, backdoors	Internal systems	Mass breach, fraud
Social engineering	Phishing, vishing, pretexting	Users/employees	Credential theft, fraud

Table 1.1: Table 1.1: Taxonomy of attack mechanisms against e-banking systems.

1.1.2 Case Studies

Case Study 1 — Bangladesh Bank Heist (2016): Attackers gained access to the Bangladesh Bank’s internal network, likely through a phishing email, and installed malware that monitored SWIFT Alliance Access software. Over several days, the attackers sent 35 fraudulent SWIFT transfer requests totaling 951million. *While most transfers were blocked, 81million was successfully transferred to accounts in the Philippines.* This case illustrates the catastrophic potential of server-server communication compromise when authentication and integrity controls on inter-bank messaging are insufficient [24][20][22].

Case Study 2 — Capital One Data Breach (2019): A former cloud services employee exploited a misconfigured web application firewall to access an AWS S3 bucket containing personal data of over 100 million credit card applicants. The breach demonstrates that cloud-hosted e-finance systems introduce new attack vectors related to identity and access management misconfigurations, and that data-at-rest encryption alone is insufficient without proper access controls [21][22].

Case Study 3 — Credential Stuffing Campaigns (2018–2024): Multiple major banks reported surges in account takeover fraud resulting from credential stuff-

ing attacks using billions of leaked username-password pairs from non-financial breaches. Despite banks implementing account lockout and anomaly detection, the sheer volume of attempts (sometimes exceeding 50 million per day across the industry) results in a non-trivial success rate. These campaigns highlight the inadequacy of password-only authentication and the need for device-bound cryptographic authentication.

Case Study 4 — EMV Relay Attacks (2020–2023): Researchers demonstrated relay attacks against contactless payment cards, where a proxy device near the victim’s card relays NFC communication to a accomplice’s device at a point-of-sale terminal hundreds of kilometers away. While EMV cryptography prevents card cloning, the relay attack bypasses proximity assumptions, showing that authentication protocols must bind cryptographic proofs to physical context [16].

Figure 1.1: Attack lifecycle in e-banking systems

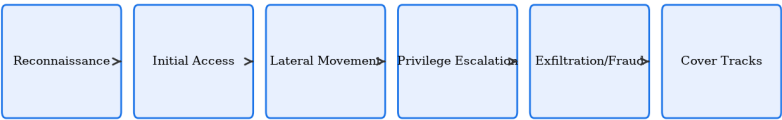


Figure 1.1: Attack lifecycle flowchart: reconnaissance <<< PH0 >>> initial access <<< PH1 >>> lateral movement <<< PH2 >>> privilege escalation <<< PH3 >>> data exfiltration/fraud <<< PH4 >>> cover tracks (Figure 1.1).

1.1.3 Attack Trends and Statistics

The financial services sector consistently ranks among the top three industries targeted by cyberattacks. The IBM Cost of a Data Breach Report (2024) reports that the average cost of a data breach in the financial sector is *5.9million, compared to across – industry average of 4.45 million*. Phishing remains the most common initial access vector, accounting for approximately 41% of breaches in financial services according to the Verizon Data Breach Investigations Report (2024) [21][22].

The adoption of mobile banking has shifted attack patterns: mobile malware targeting banking applications grew by 32% year-over-year, with trojans such as Anubis, Cerberus, and SharkBot employing overlay attacks to capture credentials

and intercept SMS OTPs. Concurrently, API attacks against open-banking endpoints have emerged as a new threat category, with the OWASP API Security Top 10 identifying broken authentication and excessive data exposure as the most prevalent vulnerabilities [23].

Ransomware groups have increasingly targeted financial institutions, with groups such as LockBit, ALPHV/BlackCat, and Cl0p claiming attacks on banks, insurance companies, and payment processors. The double-extortion model—encrypting data and threatening to publish stolen records—amplifies the pressure on victims to pay.

From a defensive standpoint, understanding attack trends enables prioritization of security investments. The shift from perimeter-focused defenses to zero-trust architectures reflects the recognition that network boundaries have dissolved. In a zero-trust model, every access request—regardless of origin—is authenticated, authorized, and encrypted. The cryptographic schemes proposed in this dissertation align with zero-trust principles by providing device-bound, transaction-scoped authentication (ECC-DTB-AKA) and persistent integrity proofs that survive service-boundary crossings (ECC-HISE).

1.1.4 Regulatory and Standards Landscape

The regulatory environment surrounding e-banking security provides both constraints and guidance for cryptographic system design. The Payment Card Industry Data Security Standard (PCI DSS) v4.0 mandates strong cryptography for cardholder data protection, requiring TLS 1.2 or higher for transmission and AES-256 for storage. The European Union’s Revised Payment Services Directive (PSD2) requires strong customer authentication (SCA) for electronic payments, implemented through two of three factors: knowledge (password), possession (device/token), and inherence (biometric) [2][17][18][42].

NIST Special Publication 800-52 Rev. 2 provides guidelines for TLS implementations in government and commercial systems, recommending TLS 1.3 with specific cipher suites. NIST SP 800-57 Part 1 offers key-management guidance, recommending ECC P-256 for protection through 2030 and P-384 for longer-term requirements. ISO/IEC 27001:2022 specifies information security management system requirements applicable to financial institutions globally [1][19][28].

The SWIFT Customer Security Programme (CSP) mandates 32 mandatory and

advisory controls for institutions connected to the SWIFT network, including multi-factor authentication for operator sessions, network segmentation, and attestation of security compliance annually. Non-compliance can result in reporting to regulators and potential disconnection from the SWIFT network [20][24].

These standards collectively establish minimum baselines but do not prescribe integrated methodologies for binding authentication to key exchange or for hierarchical data-protection envelopes. The gap between compliance and comprehensive security motivates the research contributions of this dissertation [8][29].

1.1.5 Detailed Analysis of Selected Attack Vectors

To provide depth beyond the taxonomy presented in Section 1.1.1, this subsection examines three high-impact attack vectors in detail: man-in-the-middle attacks on banking sessions, session hijacking via token theft, and advanced persistent threats (APTs) targeting financial infrastructure.

Man-in-the-middle (MITM) attacks on banking sessions exploit the fact that users often cannot distinguish legitimate bank servers from impersonating proxies. In a typical MITM scenario, the attacker positions a proxy between the client and the bank server. If the client does not validate the server’s certificate chain correctly—or if the attacker has compromised a trusted CA or uses a corporate SSL-inspection proxy—the attacker can decrypt, inspect, and modify transaction data in real time. Tools such as mitmproxy and Burp Suite, while designed for legitimate security testing, demonstrate the feasibility of this attack class. Banking trojans such as Zeus and Emotet have historically automated MITM against banking websites by injecting fraudulent transaction fields into otherwise legitimate sessions, redirecting wire transfers to attacker-controlled accounts without altering the confirmation screen displayed to the user [4].

Session hijacking via token theft represents a distinct but related threat. After successful authentication, the banking application issues a session token (cookie or bearer token) that the client presents with each subsequent request. If an attacker obtains this token—through XSS, malware memory scraping, or network sniffing on unencrypted channels—the attacker can impersonate the victim without knowing their password or MFA code. The OWASP Session Management Cheat Sheet recommends token rotation, binding tokens to client IP addresses, and using HttpOnly/Secure cookie flags, but these mitigations are not universally implemented

and do not provide cryptographic binding between the token and the underlying key-exchange material [23].

Advanced persistent threats (APTs) targeting financial infrastructure operate over extended time periods (weeks to months), maintaining covert access to bank networks while exfiltrating data or preparing fraudulent transactions. APT groups such as Carbanak (also known as FIN7) and Lazarus Group have demonstrated the ability to compromise bank internal networks, learn internal transaction workflows, and execute fraudulent transfers through the same channels used by legitimate operators. APT attacks often begin with spear-phishing emails containing weaponized documents, progress through lateral movement using stolen credentials, and culminate in access to SWIFT terminals or core banking systems. The Bangladesh Bank heist exemplifies an APT-style operation against financial infrastructure [24][25][20][22].

1.1.6 Impact Assessment Framework

Assessing the impact of attacks on e-banking systems requires a multidimensional framework that goes beyond simple financial loss figures. We adopt a framework with four impact dimensions: financial, operational, reputational, and regulatory [42][43].

Financial impact encompasses direct losses (fraudulent transfers, ransom payments), indirect losses (customer compensation, legal fees), and recovery costs (forensic investigation, system restoration). The Ponemon Institute and IBM jointly publish annual cost-of-breach figures; for financial services in 2024, the average total cost was \$5.9 million per breach, with an average of 258 days to identify and contain the breach.

Operational impact includes service downtime, degraded performance during attacks, and diversion of IT resources from planned projects to incident response. A DDoS attack against a major bank's online banking portal can prevent millions of customers from accessing services for hours, generating call-center surges and transaction backlogs. The operational resilience requirements of the Bank of England and the European Central Bank mandate maximum tolerable downtime thresholds for critical payment systems [42][43].

Reputational impact manifests as customer attrition, negative media coverage, and reduced stock valuation. Studies indicate that publicly traded banks experience

an average stock price decline of 5–7% in the week following a major breach announcement, with partial recovery over subsequent months. Customer trust, once eroded, requires sustained investment in transparency and security improvements to rebuild.

Regulatory impact includes fines, consent orders, and increased supervisory scrutiny. The GDPR permits fines up to 4% of global annual revenue for serious data protection violations. The U.S. Office of the Comptroller of the Currency (OCC) and the Federal Reserve have issued consent orders against banks with deficient cybersecurity programs, requiring multi-year remediation plans and independent third-party assessments.

1.2 Authentication and Key Exchange in E-Banking Systems

Authentication and key exchange form the first line of cryptographic defense in e-banking systems. Before any financial transaction can be authorized, the participating parties must establish mutual confidence in each other's identity and agree upon cryptographic keys for protecting subsequent communication. This section reviews the authentication and key-exchange mechanisms currently deployed in client-server and server-server environments of e-banking and e-finance services, analyzes their operational characteristics, and identifies limitations that motivate the scheme proposed in Chapter 2 [8][29][42][43].

1.2.1 Client-Server Authentication Mechanisms

The dominant client-server authentication model in retail e-banking comprises TLS 1.2/1.3 transport layer security combined with application-layer username-password authentication, often supplemented by a second factor (SMS OTP, hardware token, or mobile push notification). In this model, TLS provides server authentication via X.509 certificates and establishes encrypted channels, while the application layer authenticates the user independently [4][42][43][1].

The TLS 1.3 handshake protocol operates as follows: the client sends a ClientHello message specifying supported cipher suites and a key share (typically ECDHE on curve X25519 or P-256). The server responds with ServerHello, its certificate chain, and a key share. Both parties derive handshake traffic keys and

application traffic keys using HKDF. The full handshake requires one round trip (1-RTT); a subsequent resumption using pre-shared keys achieves 0-RTT, though 0-RTT data is vulnerable to replay attacks and is therefore restricted by banking applications [1][4][5][6].

Application-layer authentication typically follows the TLS handshake. The client submits credentials (username, password hash, or token) over the encrypted channel. The server validates credentials against a stored database, often using bcrypt, scrypt, or Argon2 password hashing. Upon success, the server issues a session token (commonly a JSON Web Token or opaque server-side session identifier) that the client presents with subsequent requests [1][28].

Multi-factor authentication (MFA) adds a second verification step. SMS-based OTP remains prevalent despite known vulnerabilities to SIM-swapping and SS7 protocol attacks. Time-based one-time passwords (TOTP) per RFC 6238, implemented by applications such as Google Authenticator, provide stronger security but are susceptible to real-time phishing proxies (e.g., Evilginx) that relay OTPs to the attacker. FIDO2/WebAuthn, based on public-key cryptography with hardware security keys or platform authenticators (biometrics), represents the current state of the art for phishing-resistant MFA [15][22].

Figure 1.2: Current client-server authentication flow

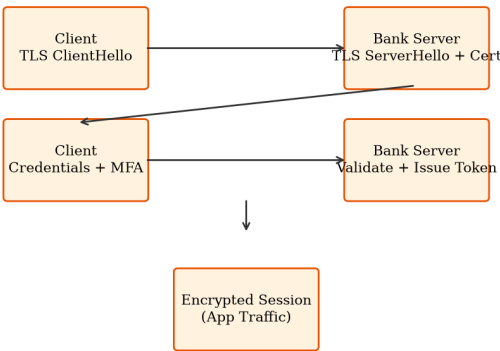


Figure 1.2: Current client-server authentication flow: TLS handshake <<< PH0 >>> credential submission <<< PH1 >>> MFA challenge <<< PH2 >>> session token issuance (Figure 1.2).

1.2.2 Server-Server Authentication and Key Exchange

Server-server authentication in e-finance encompasses communication between a bank’s internal microservices, inter-bank payment networks (SWIFT, ACH, Fedwire, SEPA), card network processors (Visa, Mastercard), and third-party open-banking aggregators. These channels typically employ mutual TLS (mTLS) with client certificates, VPN tunnels, or dedicated leased lines with link-layer encryption [1][28][4][20].

In the SWIFT network, authentication relies on the SWIFT Alliance Access software and bilateral key exchange (BKE) between institutions. Message authentication uses symmetric keys distributed through SWIFT’s Secure Channel protocol. The 2016 Bangladesh Bank heist exposed weaknesses in institutions’ local security practices surrounding SWIFT terminal access rather than fundamental flaws in SWIFT’s cryptographic protocols, but the incident nonetheless highlighted the importance of end-to-end authentication binding [8][29][24][20].

Modern microservice architectures within banks increasingly use service mesh platforms (e.g., Istio, Linkerd) that automate mTLS between services. Certificate authorities internal to the bank issue short-lived certificates (typically 24-hour validity) to each service instance. While mTLS provides strong transport security, it does not address application-level authorization, message-level integrity for stored records, or auditability requirements for regulatory compliance [1][28][4].

API-based open-banking connections (under PSD2, Open Banking UK, and similar frameworks) use OAuth 2.0 with PKCE (Proof Key for Code Exchange) for customer consent and access-token issuance. The authorization server authenticates the third-party provider, while the resource server (bank API) validates access tokens. Limitations include token theft via XSS, insufficient scope granularity, and the absence of transaction-level cryptographic binding [14][13][18][38].

1.2.3 Identified Limitations and Drawbacks

Despite the breadth of deployed mechanisms, significant limitations persist in current client-server and server-server authentication and key-exchange practices. These limitations directly motivate the ECC-DTB-AKA scheme proposed in Chapter 2.

Limitation 1 — Decoupled transport and application authentication: TLS authenticates the server (and optionally the client via mTLS) at the transport layer,

Figure 1.3: Server-server mTLS service mesh architecture

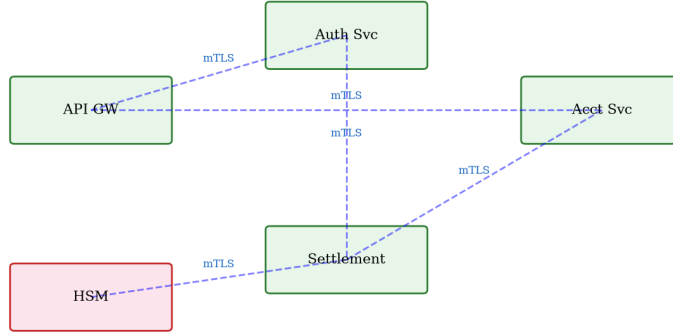


Figure 1.3: Server-server authentication in a microservice e-finance architecture with mTLS service mesh (Figure 1.3).

while user authentication occurs separately at the application layer. This decoupling creates a gap: a valid TLS session does not cryptographically bind the authenticated user identity to the encryption keys. An attacker who compromises a session token can hijack the application session even though the TLS channel remains intact [1][28].

Limitation 2 — Absence of device binding in key exchange: Standard TLS ECDHE derives session keys from ephemeral key pairs only, without incorporating device characteristics. If a session token is exfiltrated, it can be replayed from a different device. FIDO2 addresses this for the authentication step but does not integrate device binding into the key-exchange derivation itself [8][29][12][15].

Limitation 3 — No transaction-level key derivation: Once a session is established, the same application traffic keys protect all transactions within the session. Compromise of session keys exposes all transactions, not merely the current one. Per-transaction key derivation without additional round trips is not supported by TLS or OAuth 2.0 [5][6][13][14].

Limitation 4 — Session token vulnerability: Opaque session tokens and JWTs are bearer credentials—whoever possesses the token is authenticated. Token theft via XSS, malware, or network interception remains a prevalent attack vector. Binding tokens to cryptographic key material established during authenticated key agreement would limit the impact of token compromise [7][8].

Limitation 5 — Server-server channels lack message-level security: mTLS protects data in transit between services but does not provide persistent integrity proofs for stored or forwarded messages. If a message is intercepted after TLS termination and before re-encryption, it is vulnerable to tampering [1][28].

Limitation 6 — Insufficient forward secrecy in legacy deployments: Although TLS 1.3 mandates forward secrecy, many e-banking systems still support TLS 1.2 with cipher suites that do not use ephemeral Diffie-Hellman (e.g., RSA key transport). Legacy load balancers and API gateways may terminate TLS and re-encrypt with weaker configurations, creating forward-secrecy gaps [1][8][33][42].

These six limitations collectively demonstrate that current authentication and key-exchange practices in e-banking systems, while adequate for baseline compliance, fail to provide the cryptographic binding of identity, device, and transaction context necessary to resist the attack mechanisms cataloged in Section 1.1. Chapter 2 addresses these limitations through the ECC-DTB-AKA protocol, which integrates mutual authentication, ECDH-based forward secrecy, device fingerprint binding, and on-demand transaction key derivation into a unified three-round handshake [1][8][5][6].

Limitation	Affected Mechanism	Attack Enabled	Severity
Decoupled auth layers	TLS + app login	Session hijacking	High
No device binding	TLS ECDHE	Credential replay from new device	High
No per-txn key derivation	TLS session keys	Bulk transaction exposure	Medium
Bearer session tokens	JWT / opaque tokens	Token theft (XSS, malware)	High
No message-level integrity	mTLS only	Post-termination tampering	Medium
Legacy TLS configurations	TLS 1.2 RSA suites	Retroactive decryption	Medium

Table 1.2: Table 1.2: Limitations of current authentication and key-exchange mechanisms.

1.2.4 Elliptic Curve Cryptography in Financial Authentication

Elliptic curve cryptography has become the preferred public-key primitive for new financial authentication deployments. Understanding its role in current systems is essential context for the novel schemes proposed in Chapters 2 and 3 [11][34][35].

An elliptic curve E over a finite field F_p is the set of points (x, y) satisfying the Weierstrass equation $y^2 = x^3 + ax + b \pmod{p}$, together with a point at infinity

forming an abelian group. Cryptographic security relies on the hardness of the elliptic curve discrete logarithm problem (ECDLP): given points P and $Q = [k]P$, find scalar k . For properly chosen curves with sufficiently large prime subgroups, the best known algorithms require exponential time, providing security proportional to half the bit length of the subgroup order [11][34][35].

In TLS 1.3, the default key-exchange mechanism is ECDHE (Elliptic Curve Diffie-Hellman Ephemeral), using curves X25519 or P-256. The ephemeral nature of ECDHE provides forward secrecy: compromise of the server's long-term private key does not enable decryption of previously recorded sessions. EMV chip cards use ECDSA on curve secp256r1 for offline and online transaction authentication. FIDO2/WebAuthn employs ECDSA or Ed25519 for credential key pairs stored in hardware authenticators [1][8][9][26].

Despite widespread ECC adoption, current deployments use ECC primarily for individual cryptographic operations (key agreement, signing) rather than as an integrated methodology binding authentication, key exchange, device context, and transaction scope. The ECC-DTB-AKA scheme proposed in Chapter 2 extends ECC key agreement with additional binding inputs (device fingerprint, transaction nonce) in the HKDF derivation, creating a cohesive protocol rather than a stack of independent cryptographic layers [5][6][8][29].

1.2.5 Comparative Overview of Authentication Protocols

Table 1.4 provides a comparative overview of authentication and key-exchange protocols commonly used in e-banking systems, highlighting properties relevant to the limitations identified in Section 1.2.3 [42][43].

TLS 1.3 full handshake provides server authentication and forward-secure key agreement in one round trip but does not authenticate the user at the cryptographic layer. OAuth 2.0 with PKCE adds authorization delegation for third-party API access but relies on bearer tokens vulnerable to theft. FIDO2 provides phishing-resistant user authentication but does not integrate with key-exchange derivation. Kerberos with PKINIT (public-key initialization) supports mutual authentication in enterprise environments but is rarely deployed in retail e-banking client-server channels [1][14][13][15].

None of these protocols simultaneously achieves: (a) mutual authentication at the cryptographic layer, (b) device binding in key derivation, (c) per-transaction key

derivation without additional round trips, and (d) three-round handshake efficiency. This gap is the design target for ECC-DTB-AKA [5][6].

Protocol	Mutual Auth	Forward Secrecy	Device Bind	Per-Txn Key	Rou
TLS 1.3	Server only	Yes	No	No	1
OAuth 2.0 PKCE	Delegated	Via TLS	No	No	3+
FIDO2/WebAuthn	User auth	N/A	Partial	No	2
ECDH + JWT	App layer	Yes	No	No	2
ECC-DTB-AKA (Ch.2)	Yes	Yes	Yes	Yes	3

Table 1.3: Table 1.4: Comparative overview of authentication protocols for e-banking.

1.2.6 Password-Based Authentication Vulnerabilities

Despite decades of security guidance, password-based authentication remains the default for retail e-banking in most jurisdictions. Understanding its inherent vulnerabilities is essential for motivating cryptographic alternatives [42][43].

Human-chosen passwords exhibit highly skewed distributions. Analysis of breached password databases consistently shows that the top 100 passwords account for approximately 17% of all passwords, with '123456', 'password', and 'qwerty' appearing in virtually every corpus. Password complexity rules (requiring uppercase, numbers, symbols) often result in predictable substitutions (P@ssw0rd) that provide minimal additional entropy.

Password hashing algorithms mitigate offline attacks against stolen databases. Bcrypt, scrypt, and Argon2 are designed to be computationally expensive, limiting the rate at which attackers can test guessed passwords. However, weak passwords remain vulnerable even with strong hashing, and credential stuffing attacks bypass hashing entirely by testing leaked credentials from other breaches against banking login portals.

Password reset mechanisms introduce additional attack surfaces. Knowledge-based authentication questions (mother's maiden name, first pet) are often guessable from social media. Email-based reset links can be intercepted if the email account is compromised. SMS-based reset codes inherit SIM-swapping vulnerabilities. These weaknesses reinforce the need for cryptographic authentication mechanisms that do not rely solely on shared secrets memorized by humans.

1.3 Data Encryption, Security, and Integrity in E-Banking

Beyond authentication and key establishment, e-banking systems must protect the confidentiality, integrity, and availability of financial data throughout its lifecycle. This section examines the data encryption, security, and integrity mechanisms currently employed in e-banking and e-finance services, with emphasis on server-server data transfer and storage, and identifies drawbacks that motivate the ECC-HISE scheme proposed in Chapter 3 [42][43].

1.3.1 Data Encryption Mechanisms

Data-at-rest encryption in banking systems typically employs AES-256 in Galois/Counter Mode (GCM) or XTS mode. Database-level encryption (transparent data encryption, TDE) protects entire tablespaces, while column-level encryption targets sensitive fields (account numbers, social security numbers). Key management is performed by HSMs certified to FIPS 140-2 Level 3 or higher, with key-encryption keys (KEKs) stored in hardware and data-encryption keys (DEKs) wrapped under KEKs [2][3].

Data-in-transit encryption relies primarily on TLS as described in Section 1.2. For server-server bulk data transfer (e.g., end-of-day settlement files, regulatory reporting batches), institutions may additionally employ PGP/GPG encryption or proprietary file-encryption tools with pre-shared symmetric keys distributed via out-of-band channels [1][28].

Elliptic curve cryptography appears in data-protection contexts primarily through ECIES (Elliptic Curve Integrated Encryption Scheme), which combines ECDH key agreement with a symmetric cipher and MAC. ECIES is standardized in IEEE 1363a and SEC 1, and provides semantic security under chosen-ciphertext attacks when implemented with appropriate KDF and MAC (or authenticated encryption). However, standard ECIES operates at the message level without hierarchical key management or batch audit capabilities required for inter-bank settlement [36][2][3][41].

1.3.2 Data Integrity and Audit Mechanisms

Integrity protection for individual messages is commonly provided by HMAC-SHA256 or by the authenticated encryption inherent in AES-GCM. Digital signatures using RSA or ECDSA provide non-repudiation for high-value transactions

Figure 1.4: Data protection layers in e-banking systems

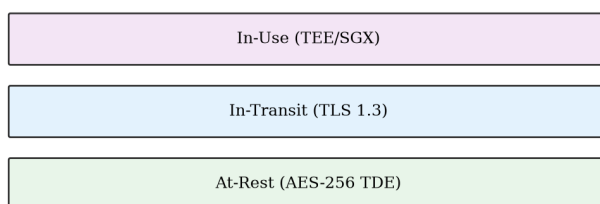


Figure 1.4: Data protection layers in e-banking: at-rest (TDE), in-transit (TLS), in-use (confidential computing) (Figure 1.4).

and regulatory submissions. However, per-message signatures do not efficiently support batch verification of large settlement files containing millions of records [2][36][9][26].

Audit trails in e-banking systems log transaction events, access attempts, and administrative actions to tamper-evident storage. Common implementations include append-only databases, write-once-read-many (WORM) storage, and centralized SIEM platforms. Blockchain and distributed-ledger technologies have been proposed and piloted for settlement audit (e.g., JPMorgan’s Onyx, R3 Corda), offering Merkle-tree-based integrity guarantees but introducing latency and scalability trade-offs unsuitable for all use cases [30][32][31][42].

Hash chains—where each log entry includes the hash of the previous entry—provide lightweight tamper evidence without the overhead of full blockchain consensus. Merkle trees enable efficient verification that a specific record is included in a batch without reprocessing the entire batch, a property essential for regulatory audit of large settlement files [30][31][32].

1.3.3 Identified Limitations and Drawbacks

The following limitations in current data encryption, security, and integrity mechanisms motivate the ECC-HISE scheme proposed in Chapter 3.

Limitation 1 — Flat key management hierarchies: Many institutions use a small number of long-lived symmetric keys for bulk encryption, increasing the blast radius of key compromise. Hierarchical key derivation that scopes keys to business

domains and individual batches limits exposure [3][5][6].

Limitation 2 — Disconnected encryption and audit: Encryption and audit logging are typically implemented as independent systems. An encrypted batch file may be stored without a cryptographically linked integrity proof that enables third-party verification of completeness and ordering.

Limitation 3 — Inefficient batch integrity verification: Per-record digital signatures do not scale to settlement batches containing millions of entries. A Merkle-tree-based approach enables $O(\log n)$ verification per record while maintaining $O(1)$ batch-level integrity evidence [26][9][30].

Limitation 4 — Static pre-shared keys in server-server channels: Inter-bank file transfer often relies on symmetric keys distributed manually and rotated infrequently (annually or biannually). ECC-based ephemeral key agreement per batch eliminates long-lived shared secrets.

Limitation 5 — Lack of end-to-end integrity across service boundaries: In microservice architectures, a message may be decrypted at each service boundary (TLS termination), processed in plaintext, and re-encrypted. Envelope encryption that persists integrity proofs across service boundaries addresses this gap [1][28].

Limitation 6 — Regulatory audit friction: Demonstrating to auditors that no records were added, removed, or reordered in a settlement batch currently requires manual reconciliation or expensive full-batch reprocessing. Cryptographic audit chains reduce audit cost and increase confidence.

These six limitations in data-security mechanisms parallel the authentication gaps identified in Section 1.2, but operate at a different layer of the system stack. While Section 1.2 concerns who is communicating and how keys are established, this section concerns what data is protected, how integrity is maintained, and how compliance can be demonstrated. Chapter 3 addresses these limitations through ECC-HISE, which combines hierarchical ECC key derivation, authenticated envelope encryption, and Merkle-tree audit chains into a cohesive server-server data-protection methodology [5][6][30].

1.3.4 Key Management in Financial Institutions

Effective data encryption in e-banking depends critically on key management—the generation, distribution, storage, rotation, and destruction of cryptographic keys. Poor key management can render even the strongest encryption algorithms

Limitation	Current Practice	Proposed Direction (Ch.3)	Impact
Flat key hierarchy	Few long-lived DEKs	ECC hierarchical derivation	Key compromise
Disconnected audit	Separate log systems	Merkle-linked envelopes	Tamper detection
Batch verification cost	Per-record signatures	Merkle tree batch proofs	Scalability
Static PSKs	Annual manual rotation	Ephemeral ECDH per batch	Long-term exposure
Service-boundary gaps	TLS terminate/decrypt	Persistent envelope integrity	Insider tampering
Audit friction	Manual reconciliation	Cryptographic audit chain	Compliance cost

Table 1.4: Table 1.3: Limitations of current data security mechanisms and proposed directions.

ineffective, as demonstrated by numerous breaches where encryption was deployed but keys were stored alongside encrypted data in accessible locations [3][42][43].

Financial institutions typically employ a tiered key-management architecture. At the top level, master keys are generated within FIPS 140-2 Level 3 (or higher) hardware security modules and never leave the HSM in plaintext form. Key-encryption keys (KEKs) wrap data-encryption keys (DEKs) for distribution to application servers. DEKs encrypt actual customer data and are rotated frequently (daily to monthly) depending on data classification and regulatory requirements.

The ANSI X9.24-3 standard specifies key-management requirements for symmetric keys in financial services, including dual-control procedures for key generation and split-knowledge requirements for critical keys. PCI DSS Requirement 3 mandates protection of stored cardholder data, including key-management procedures for encryption keys. However, these standards focus on symmetric key hierarchies and do not address ECC-based hierarchical derivation for batch envelope encryption with integrated audit chains—the design space targeted by ECC-HISE [17].

1.3.5 Merkle Trees and Cryptographic Audit Trails

Merkle trees (hash trees) provide an efficient data structure for verifying integrity of large datasets. A Merkle tree is a binary tree in which each leaf node contains the hash of a data block, and each internal node contains the hash of the concatenation of its children. The root hash commits to the entire dataset: any modification to any leaf changes the root hash. A verifier can confirm inclusion of a specific record with $O(\log n)$ hash computations given an authentication path (Merkle proof) from

the leaf to the root [30].

In e-banking settlement, Merkle trees enable regulators and counterparties to verify that a specific transaction was included in a batch settlement without reprocessing the entire batch. Combined with digital signatures on the Merkle root, the tree provides both integrity and non-repudiation at the batch level. Bitcoin and other blockchains use Merkle trees in block headers for similar reasons, though blockchain consensus introduces latency unsuitable for real-time settlement. ECC-HISE adopts Merkle trees for audit without requiring distributed consensus, making it applicable to traditional inter-bank settlement workflows [26][9][30][31].

The integration of Merkle audit chains with ECC-based envelope encryption—rather than treating encryption and audit as separate subsystems—constitutes a key differentiator of the ECC-HISE approach developed in Chapter 3. By including the Merkle root in the signed batch header, any tampering with individual records is detectable through root mismatch, while the hierarchical key structure limits the scope of key compromise to individual batches within specific business domains [30].

1.3.6 Homomorphic Encryption and Emerging Approaches

While not part of the proposed schemes in this dissertation, emerging cryptographic technologies warrant brief discussion to situate the research within the broader landscape of financial data protection.

Fully homomorphic encryption (FHE) enables computation on encrypted data without decryption, potentially allowing banks to perform fraud detection on encrypted transaction streams. However, FHE remains orders of magnitude slower than plaintext computation and is not yet practical for real-time e-banking workloads. Partial homomorphic schemes (Paillier, ElGamal) support specific operations (addition, multiplication) and have been piloted for privacy-preserving credit scoring [42][43].

Confidential computing (Intel SGX, AMD SEV, ARM TrustZone) provides hardware-enforced memory isolation for applications processing sensitive data. Major cloud providers offer confidential virtual machines for financial workloads. These technologies address the 'data in use' protection gap identified in Section 1.3.1 but require trust in hardware manufacturers and are vulnerable to side-channel attacks.

The schemes proposed in this dissertation (ECC-DTB-AKA and ECC-HISE) complement rather than replace these emerging approaches. ECC-DTB-AKA secures the client-server channel; ECC-HISE secures server-server batch data; confidential computing can protect in-memory processing; and post-quantum algorithms will eventually replace specific ECC primitives as standards mature [39].

1.4 Threat Model, Assumptions, and Research Questions

This section formalizes the threat model, system assumptions, and research questions that guide the design and evaluation of the proposed schemes in Chapters 2 through 5.

1.4.1 Threat Model

We adopt a Dolev-Yao threat model extended with financial-domain adversary capabilities. The adversary A can: (a) eavesdrop on, insert, delete, modify, and replay messages on all network channels; (b) compromise up to $N-1$ of M server nodes in the settlement network (where $M > 2N-1$ for quorum systems, but we do not assume quorum); (c) extract session tokens and application data from a compromised client device; (d) perform offline dictionary attacks on captured password hashes (but not on ECDH ephemeral private keys due to computational hardness); (e) submit fraudulent transaction requests using stolen credentials [12][29][37].

The adversary A cannot: (a) break the elliptic curve discrete logarithm problem (ECDLP) or compute discrete logarithms on the chosen curve; (b) break SHA-256, AES-256-GCM, or Ed25519 unforgeability; (c) extract private keys from an HSM (in the prototype, HSM is simulated and this assumption is noted as a limitation); (d) physically access the bank's data center without detection [2][10][11][34].

Trust assumptions: the bank's certificate authority (CA) is trusted; registered client public keys are distributed securely during account enrollment; the clearing-house master key is generated and stored in an HSM at setup time [4].

1.4.2 System Assumptions

The e-banking system under consideration comprises: (1) a client application running on a smartphone or desktop with a secure element or trusted execution environment for key storage; (2) a bank application server providing RESTful

Figure 1.5: Threat model for e-banking system

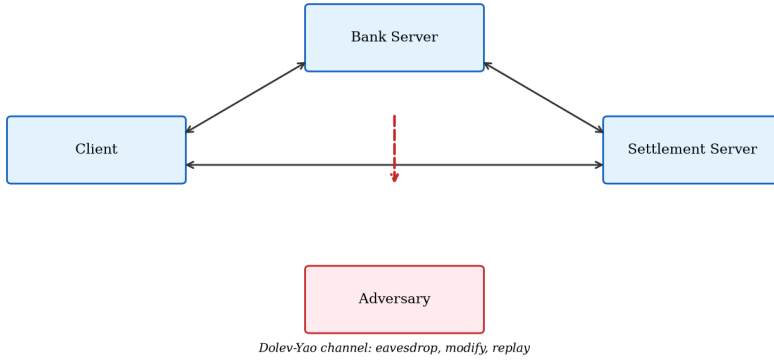


Figure 1.5: Threat model diagram showing adversary position relative to client, bank server, and inter-bank network (Figure 1.5).

APIs for account management and transaction authorization; (3) a settlement server participating in inter-bank batch transfer; (4) a clearing-house authority managing the master key hierarchy [42][43].

Clients and servers have synchronized clocks within ± 5 minutes (for timestamp validation and nonce freshness). Network connectivity is intermittent but eventually available. The elliptic curve used is SECP256R1 (NIST P-256) for ECDH/ECDSA operations and Ed25519 for batch signatures, unless otherwise noted [9][26][10][11].

1.4.3 Research Questions

RQ1: Can an ECC-based authenticated key agreement protocol that binds session keys to device fingerprints and transaction nonces overcome the authentication limitations identified in Section 1.2 while maintaining or improving communication efficiency? [7][8]

RQ2: Can a hierarchical ECC-based secure envelope scheme with Merkle audit chains address the data-security limitations identified in Section 1.3 for server-server batch transfer in e-finance systems? [30]

RQ3: How do the proposed schemes compare quantitatively to existing approaches (TLS 1.3, standard ECDH, ECIES, static PSK) in terms of authentication

latency, ciphertext overhead, communication round trips, and resistance to attacks cataloged in Section 1.1? [1][41][12][29]

RQ4: Can both schemes be implemented in a unified e-financial service architecture and validated through programmatic testing and benchmark measurement?

1.4.4 Chapter Summary

This chapter has defined and categorized attack mechanisms against e-banking and e-finance systems, presented case studies demonstrating their real-world impact, reviewed current authentication and key-exchange mechanisms with six identified limitations (Section 1.2), reviewed current data encryption and integrity mechanisms with six identified limitations (Section 1.3), and formalized the threat model and research questions guiding this dissertation. The identified limitations provide direct motivation for the two novel ECC-based schemes developed in Chapters 2 and 3, whose prior-work context is further elaborated in Chapter 5 [42][43].

1.4.5 Security Requirements Derivation

From the threat model (Section 1.4.1) and the identified limitations (Sections 1.2.3 and 1.3.3), we derive the following security requirements that the proposed schemes must satisfy. These requirements serve as evaluation criteria in Chapter 5.

SR1 (Mutual Authentication): Both communicating parties must cryptographically verify each other's identity before any financial data is exchanged. For client-server channels, this means the client verifies the bank server's identity via certificate chain, and the server verifies the client's identity via registered public key signature [4].

SR2 (Forward Secrecy): Compromise of long-term private keys must not enable decryption of previously recorded sessions. Ephemeral ECDH key pairs must be used for session key establishment [1][8][12][29].

SR3 (Device Binding): Session keys must be cryptographically bound to the client device, preventing session transfer to unauthorized devices even if session tokens are exfiltrated.

SR4 (Transaction Binding): Individual transactions must be protected by keys derived specifically for that transaction, limiting the impact of single-transaction key compromise.

SR5 (Data Confidentiality): Financial records in server-server batch transfer must be encrypted with semantically secure authenticated encryption [36][2].

SR6 (Data Integrity): Any modification to batch records must be detectable through cryptographic verification, with batch-level integrity proofs enabling efficient audit.

SR7 (Non-Repudiation): Batch senders must not be able to deny having sent a batch, achieved through digital signatures on batch headers [26][9].

SR8 (Efficiency): Authentication must complete within acceptable latency bounds (target: under 200ms on commodity hardware) with minimal communication overhead.

1.4.6 Relationship to Subsequent Chapters

The security requirements SR1 through SR4 are addressed primarily by the ECC-DTB-AKA protocol developed in Chapter 2. Requirements SR5 through SR7 are addressed by the ECC-HISE scheme developed in Chapter 3. Requirement SR8 is evaluated quantitatively in Chapters 2, 3, and 5 through benchmark measurements of the prototype described in Chapter 4.

Chapter 5 provides the comprehensive prior-work analysis specified in the dissertation requirements: Section 5.2 reviews authentication and key-exchange literature in depth (expanding on the overview in Section 1.2), Section 5.3 reviews data encryption and integrity literature (expanding on Section 1.3), and Section 5.4 synthesizes findings to demonstrate how the proposed schemes address the vulnerabilities cataloged in this chapter.

Readers approaching this dissertation from a practitioner perspective may wish to read Section 1.1 and Sections 1.2.3/1.3.3 first to understand the threat landscape and identified gaps, then proceed to Chapters 2 and 3 for protocol specifications, and Chapter 4 for implementation guidance. Readers with a research orientation may prefer to read Chapter 5’s prior-work analysis before the proposed schemes, to situate the contributions within the existing literature.

Chapter 2

ECC-DTB-AKA Authenticated Key Agreement

ECC-DTB-AKA: Device- and Transaction-Bound Authenticated Key Agreement for Client-Server E-Banking Channels

2.1 Introduction and Motivation

Chapter 1 identified six critical limitations in current authentication and key-exchange mechanisms deployed in e-banking client-server channels (Section 1.2.3). These limitations—decoupled transport and application authentication, absence of device binding, lack of per-transaction key derivation, bearer-token vulnerability, insufficient message-level integrity across service boundaries, and legacy TLS configurations without forward secrecy—collectively create exploitable gaps that attackers leverage through session hijacking, credential replay, and man-in-the-middle attacks [1][8][5][6].

This chapter proposes ECC-DTB-AKA (Elliptic Curve Cryptography — Device- and Transaction-Bound Authenticated Key Agreement), an alternative approach to strengthening security systems in e-banking and e-finance environments. ECC-DTB-AKA integrates mutual authentication, ephemeral ECDH key agreement, device fingerprint binding, and on-demand transaction key derivation into a unified three-round protocol. By incorporating device and transaction context directly into the HKDF key-derivation salt, the scheme ensures that session keys and transaction

keys are cryptographically bound to the originating device and specific financial operation, addressing limitations that existing TLS-centric and OAuth-based approaches cannot resolve without additional protocol layers [5][6][7][8].

The design philosophy of ECC-DTB-AKA follows the principle of cryptographic cohesion: rather than stacking independent security layers (TLS for transport, JWT for session, OTP for transaction), all authentication and key-establishment functions are unified within a single ECC-based protocol. This reduces the attack surface at layer boundaries and provides formal security properties that can be analyzed within a single proof framework [1][28].

2.1.1 Limitations of Layered Security Architectures

Contemporary e-banking systems typically implement security as a vertical stack of independent layers: network firewalls at the perimeter, TLS at the transport layer, web application firewalls at the HTTP layer, authentication middleware at the application layer, and database encryption at the persistence layer. While defense-in-depth is a sound principle, the interfaces between layers create implicit trust assumptions that attackers exploit [42][43][1][28].

Consider a typical mobile banking login flow: (1) the mobile app establishes a TLS 1.3 connection to the API gateway, authenticating the server via its certificate; (2) the user enters credentials, which the app transmits over the TLS channel; (3) the authentication service validates credentials and returns a JWT session token; (4) subsequent API calls include the JWT in the Authorization header, still over TLS. At each layer boundary, data transitions between protection domains. The TLS layer knows nothing about the user's identity; the JWT layer knows nothing about the encryption keys; the transaction layer knows nothing about the originating device [1][4][28].

An attacker who compromises the JWT (via malware, XSS, or network interception on a misconfigured endpoint) gains full session access without breaking TLS. An attacker who performs SSL stripping at a captive portal downgrades the transport layer while the application layer continues to function, potentially exposing credentials. ECC-DTB-AKA collapses layers 1–3 into a single cryptographic protocol where user identity, device context, and session keys are established atomically [1][28].

2.2 Design Goals and Requirements

Derived from the security requirements SR1–SR4 and SR8 established in Section 1.4.5, ECC-DTB-AKA is designed to satisfy the following goals:

G1 (Mutual Authentication): Both client and server must cryptographically prove their identity during key establishment, not merely after an encrypted channel is created.

G2 (Forward Secrecy): Compromise of long-term private keys must not enable retroactive decryption of previously established sessions [1][8].

G3 (Device Binding): Session keys must incorporate a device fingerprint in the derivation function, preventing session transfer to unauthorized devices.

G4 (Transaction Binding): Per-transaction keys must be derivable from the session master key without additional communication rounds.

G5 (Efficiency): Authentication must complete with minimal latency and message overhead, suitable for mobile banking on cellular networks.

G6 (Standards Alignment): The protocol must use NIST-approved curves (SECP256R1), SHA-256, HKDF (RFC 5869), and ECDSA (FIPS 186-5), enabling integration with existing PKI and HSM infrastructure [5][6][9][26].

2.2.1 Elliptic Curve Cryptography Foundations

The selection of elliptic curve cryptography as the sole public-key primitive in ECC-DTB-AKA warrants explicit justification given the availability of alternatives (RSA, DSA, post-quantum algorithms) [11][34][35][39].

Elliptic curves over finite fields provide a group structure in which the Diffie-Hellman problem is hard: given G , $[a]G$, $[b]G$, computing $[ab]G$ is computationally infeasible. The security level of SECP256R1 (NIST P-256) is approximately 128 bits, equivalent to AES-128 and SHA-256. The key size of 256 bits compares favorably to 3072-bit RSA keys for equivalent security, reducing bandwidth, storage, and computation [11][34][35][27].

SECP256R1 is specified in FIPS 186-5 and SP 800-186, widely implemented in hardware (HSMs, smart cards, TPMs), and supported by every major TLS library. Its curve parameters were generated verifiably (though debates about potential backdoors in NIST curves persist in the literature; Curve25519 is an alternative that ECC-DTB-AKA could adopt with minimal protocol changes). For e-banking

deployments requiring FIPS compliance, SECP256R1 is the pragmatic choice [40][26][42][43].

ECDSA signatures on SECP256R1 provide the authentication mechanism in ECC-DTB-AKA. Each signature operation requires one scalar multiplication and one modular inversion, typically completing in under 1 ms on modern processors. The prototype measurements confirm that four signature operations per handshake (two per party) dominate the computational cost, accounting for approximately 80% of total authentication latency [9][26].

2.3 Protocol Specification

This section provides the complete specification of ECC-DTB-AKA, including notation, algorithmic descriptions, and message formats. The protocol operates between a banking client C and banking server S over an insecure channel.

2.3.1 Notation and Prerequisites

Let $E/\mathbb{F}_p$ be the elliptic curve SECP256R1 with generator G and prime order q . $H: \{0,1\}^* \rightarrow \{0,1\}^{256}$ denotes SHA-256. $\text{HKDF}_{\text{extract}}$ and $\text{HKDF}_{\text{expand}}$ denote the extract and expand functions per RFC 5869. $\parallel$ denotes concatenation. C holds a long-term key pair $(\text{sk}_C, \text{PK}_C = [\text{sk}_C]G)$ registered with S during account enrollment. S holds a long-term key pair $(\text{sk}_S, \text{PK}_S = [\text{sk}_S]G)$ with an X.509 certificate issued by the bank's CA. $\text{device}_{\text{fp}} = H(\text{hardware}_{\text{attributes}})_{[128]}$ is a 128-bit device fingerprint computed from stable hardware attributes (platform ID, secure enclave identifier). Each protocol run generates ephemeral key pairs $(\text{ek}_C, \text{EK}_C = [\text{ek}_C]G)$ and $(\text{ek}_S, \text{EK}_S = [\text{ek}_S]G)$ [4][5][6][11].

2.3.2 Round 1 — Client Init (MSG1)

The client initiates the protocol by generating an ephemeral key pair $(\text{ek}_C, \text{EK}_C)$, sampling a 128-bit nonce $\text{nonce}_c \leftarrow \{0,1\}^{128}$, and computing the transcript hash $t_C = H(\text{EK}_C \parallel \text{PK}_C \parallel \text{device}_{\text{fp}} \parallel \text{nonce}_c)$. The client signs this hash with its long-term private key: $_C = \text{Sig}_{\text{sk}_C}(t_C)$. The first message is $\text{MSG1} = (\text{EK}_C, \text{PK}_C, \text{device}_{\text{fp}}, \text{nonce}_c, _C)$.

MSG1 serves three purposes: (1) the ephemeral public key EK_C enables ECDH key agreement; (2) the long-term public key PK_C and signature $_C$ authenticate the

client; (3) the device fingerprint and nonce bind the session to a specific device and prevent replay of prior MSG1 values [12][29].

Figure 2.1: ECC-DTB-AKA three-round handshake

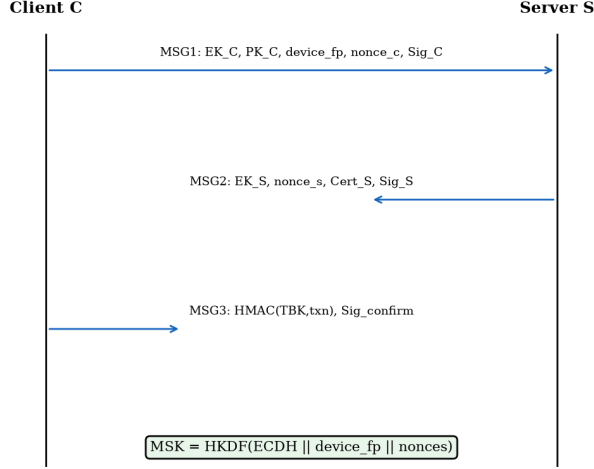


Figure 2.1: ECC-DTB-AKA three-round handshake sequence diagram (Figure 2.1).

2.3.3 Round 2 — Server Response (MSG2)

Upon receiving MSG1, the server performs the following steps: (1) Look up PK_C in the customer registry; reject if not found. (2) Verify $_C$ against t_C using PK_C ; reject if invalid. (3) Generate ephemeral (ek_S, EK_S) and sample nonce_s $\lll PH6 \ggg \{0,1\}^{128}$. (4) Compute shared secret $Z = [ek_S]EK_C$. (5) Derive master session key $MSK = HKDF(Z, device_{fp} \parallel nonce_c \parallel nonce_s, 'ECC-DTB-AKA-v1', 256)$. (6) Compute $_S = Sig_{\{sk_S\}}(H(EK_S \parallel nonce_s \parallel MSK))$. (7) Send $MSG2 = (EK_S, nonce_s, Cert_S, _S)$ [5][6].

The server signature over EK_S , $nonce_s$, and MSK provides key confirmation: the client can verify that the server computed the same MSK before proceeding. The server certificate $Cert_S$ enables the client to validate server identity through the bank's PKI chain [4].

2.3.4 Round 3 — Client Confirm (MSG3)

The client verifies Cert_S , recomputes $Z = [\text{ek}_C] \text{EK}_S$ and MSK, and validates $_S$. For a pending transaction with nonce $\text{txn}_{\text{nonce}}$, the client derives the transaction-bound key $\text{TBK}_{\text{txn}} = \text{HKDF}(\text{MSK}, \text{txn}_{\text{nonce}}, \text{'TBK'}, 256)$ and computes $\text{Sig} = \text{HMAC}(\text{TBK}_{\text{txn}}, \text{txn}_{\text{nonce}})$. The client sends $\text{MSG3} = (\text{, txn}_{\text{nonce}}, \text{Sig}_{\{\text{sk}_C\}}(\text{H}(\text{MSK} \parallel \text{'confirm'})))$ [5][6].

The server verifies the client confirmation signature, recomputes TBK_{txn} , and validates $_S$. Upon success, both parties hold MSK and can derive additional transaction keys for subsequent operations without further handshakes until session expiration.

Figure 2.2: Key derivation hierarchy in ECC-DTB-AKA

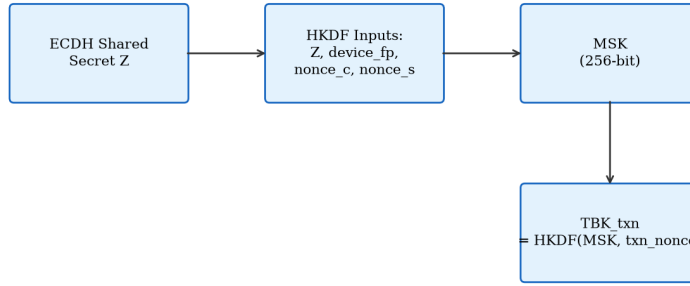


Figure 2.2: Key derivation hierarchy from ECDH shared secret to MSK and TBK_{txn} (Figure 2.2).

2.3.5 Transaction Key Derivation

A distinguishing feature of ECC-DTB-AKA is the ability to derive per-transaction keys from the established MSK without additional protocol rounds. For each transaction T_i with unique nonce $\text{txn}_{\text{nonce}_i}$, both parties compute $\text{TBK}_i = \text{HKDF}(\text{MSK}, \text{txn}_{\text{nonce}_i}, \text{'TBK'}, 256)$. Transaction data is protected using AES-256-GCM with TBK_i split into encryption and authentication subkeys. If a single transaction key is compromised, other transactions within the same session remain protected because

HKDF derivation is one-way and domain-separated by distinct $\text{txn}_{\text{nonce}}$ values [2][5][6].

2.3.6 Message Format Summary

Table 2.3 summarizes the wire-format fields for each protocol message. All integer fields are encoded in big-endian. Public keys use X9.62 uncompressed point format (65 bytes for P-256). Signatures are DER-encoded ECDSA (typically 70–72 bytes) [9][26].

The total payload for a complete handshake is approximately 569 bytes as measured in the prototype, making ECC-DTB-AKA suitable for bandwidth-constrained mobile networks. By comparison, a typical TLS 1.3 handshake with a 2 KB certificate chain exceeds 3 KB before application data [1][4][28].

Message	Field	Size (bytes)	Description
MSG1	EK_C	65	Client ephemeral public key
MSG1	PK_C	65	Client long-term public key
MSG1	$\text{device}_{\text{fp}}$	16	Device fingerprint
MSG1	nonce_c	16	Client nonce
MSG1	$_C$	~72	ECDSA signature
MSG2	EK_S	65	Server ephemeral public key
MSG2	nonce_s	16	Server nonce
MSG2	Cert_S	65	Server public key (prototype)
MSG2	$_S$	~72	ECDSA signature
MSG3		32	HMAC-SHA256 transaction tag
MSG3	$\text{txn}_{\text{nonce}}$	16	Transaction nonce
MSG3	$_confirm$	~72	Client confirmation signature

Table 2.1: Table 2.3: ECC-DTB-AKA message field specification.

2.3.7 Algorithm: Client Handshake

Algorithm 2.1 presents the client-side handshake procedure in pseudocode form. The algorithm takes as input the client’s long-term key pair (sk_C , PK_C), device attributes, and an optional transaction nonce. It outputs the three protocol messages and the established session state.

Algorithm 2.1 (ClientHandshake): (1) $\text{device}_{\text{fp}} \lll PH1 \ggg H(\text{hardware}_{\text{attributes}})[0:128]$; (2) $(\text{ek}_C, \text{EK}_C) \lll PH5 \ggg \text{KeyGen}(E)$; (3) $\text{nonce}_c \lll PH7 \ggg \{0,1\}^{128}$;

(4) $t_C \lll PH9 \ggg H(EK_C \parallel PK_C \parallel device_{fp} \parallel nonce_c)$; (5) $_C \lll PH14 \ggg ECDSA.Sign(sk_C, t_C)$; (6) $MSG1 \lll PH17 \ggg (EK_C, PK_C, device_{fp}, nonce_c, _C)$; Send($MSG1$); (7) Receive($MSG2 = (EK_S, nonce_s, Cert_S, _S)$); (8) Verify $Cert_S$ against bank CA; (9) $Z \lll PH26 \ggg ECDH(ek_C, EK_S)$; (10) $MSK \lll PH29 \ggg HKDF(Z, device_{fp} \parallel nonce_c \parallel nonce_s, 'ECC-DTB-AKA-v1')$; (11) Verify $_S = ECDSA.Sign_{\{PK_S\}}(H(EK_S \parallel nonce_s \parallel MSK))$; (12) $TBK \lll PH36 \ggg HKDF(MSK, tx_{nonce}, 'TBK')$; (13) $\lll PH38 \ggg HMAC(TBK, tx_{nonce})$; (14) $MSG3 \lll PH40 \ggg (, tx_{nonce}, ECDSA.Sign(sk_C, H(MSK \parallel 'confirm')))$; Send($MSG3$); (15) Erase ek_C ; Return (MSK, TBK).

2.3.8 Algorithm: Server Handshake

Algorithm 2.2 presents the server-side procedure. The server maintains a customer registry mapping PK_C to account metadata and validates each handshake against this registry.

Algorithm 2.2 (ServerHandshake): (1) Receive($MSG1$); (2) Look up PK_C in customer registry; abort if not found; (3) $t_C \lll PH2 \ggg H(EK_C \parallel PK_C \parallel device_{fp} \parallel nonce_c)$; (4) Verify $_C = ECDSA.Sign_{\{PK_C\}}(t_C)$; abort if invalid; (5) $(ek_S, EK_S) \lll PH11 \ggg KeyGen(E)$; (6) $nonce_s \lll PH13 \ggg \{0,1\}^{128}$; (7) $Z \lll PH14 \ggg ECDH(ek_S, EK_C)$; (8) $MSK \lll PH17 \ggg HKDF(Z, device_{fp} \parallel nonce_c \parallel nonce_s, 'ECC-DTB-AKA-v1')$; (9) $_S \lll PH21 \ggg ECDSA.Sign(sk_S, H(EK_S \parallel nonce_s \parallel MSK))$; (10) $MSG2 \lll PH25 \ggg (EK_S, nonce_s, Cert_S, _S)$; Send($MSG2$); (11) Receive($MSG3 = (, tx_{nonce}, _confirm)$); (12) Verify $_confirm = ECDSA.Sign_{\{PK_C\}}(H(MSK \parallel 'confirm'))$; (13) $TBK \lll PH31 \ggg HKDF(MSK, tx_{nonce}, 'TBK')$; (14) Verify $= HMAC(TBK, tx_{nonce})$; (15) Erase ek_S ; Return (MSK, TBK) [5][6][9][26].

2.3.9 Error Handling and Abort Conditions

The protocol defines explicit abort conditions to prevent oracle attacks and ensure fail-safe behavior. The server aborts if: (a) PK_C is not in the customer registry; (b) $_C$ verification fails; (c) $device_{fp}$ matches a blocked device list; (d) $nonce_c$ has been seen within the replay window (5 minutes). The client aborts if: (a) $Cert_S$ validation fails; (b) $_S$ verification fails; (c) MSK computed locally does not match the value implied by $_S$. Either party aborts if $MSG3$ confirmation fails.

Abort messages do not reveal which check failed, preventing attackers from using error responses as oracles for signature forgery or nonce guessing. All abort events are logged to the security operations center with timestamp, source IP, and truncated protocol identifiers for forensic analysis.

2.4 Security Analysis

This section provides a structured security analysis of ECC-DTB-AKA within the Bellare-Rogaway authenticated key exchange (AKE) model, augmented with device-binding and transaction-binding security definitions [7][8][29].

2.4.1 Security Model and Assumptions

We adopt the Canetti-Krawczyk (CK) model for authenticated key exchange, in which a protocol is secure if no polynomial-time adversary can distinguish the established session key from a random string, even with access to a key-reveal oracle (for sessions other than the test session), a signature-forgery oracle, and full control over network messages. We augment this model with a device-binding experiment: the adversary must not distinguish sessions established with $\text{device}_{\text{fp}_1}$ from those with $\text{device}_{\text{fp}_2}$ when replaying captured messages on a different device [7][8][29].

Assumptions: (A1) ECDLP is computationally hard on SECP256R1; (A2) SHA-256 is collision-resistant and preimage-resistant; (A3) HKDF is a computational PRF when IKM has sufficient entropy; (A4) ECDSA is existentially unforgeable under chosen message attack (EUF-CMA); (A5) the bank CA is trusted and client public keys are registered securely [5][6][9][26].

2.4.2 Security Properties and Proof Sketches

Theorem 2.1 (Authenticated Key Exchange Security): Under assumptions A1–A5, ECC-DTB-AKA is secure in the CK model. Proof sketch: Suppose adversary A distinguishes MSK from random. A must either break ECDH (recover Z from EK_C , EK_S without ephemeral private keys), break HKDF (distinguish MSK from random given Z), or break ECDSA (forge $_C$ or $_S$ to impersonate a party). Each reduction is standard; the ephemeral ECDH keys provide forward secrecy because MSK depends on ek_C and ek_S , which are erased after the handshake [1][8][5][6].

Theorem 2.2 (Device Binding): An adversary who captures MSG1, MSG2, and MSG3 cannot establish a valid session on a device with fingerprint $\text{device}_{\text{fp}}$, unless $\text{device}_{\text{fp}}$ is substituted in MSG1 (which invalidates $_C$) or HKDF is broken. Proof sketch: $\text{MSK} = \text{HKDF}(Z, \text{device}_{\text{fp}} \parallel \text{nonce}_c \parallel \text{nonce}_s, \dots)$. Changing $\text{device}_{\text{fp}}$ changes the salt, producing a different MSK. The server recomputes MSK from the $\text{device}_{\text{fp}}$ in MSG1, so a replay with altered $\text{device}_{\text{fp}}$ fails signature verification [5][6].

Theorem 2.3 (Transaction Binding): Given MSK, computing TBK_{txn} without knowing $\text{txn}_{\text{nonce}}$ requires breaking HKDF one-wayness. Replay of a transaction confirmation ($\text{txn}_{\text{nonce}}$) for a different transaction fails because $\text{HMAC}(\text{TBK}_{\text{txn}}, \text{txn}_{\text{nonce}})$ is verified against the specific $\text{txn}_{\text{nonce}}$ [5][6].

2.4.3 Attack Resistance Analysis

Table 2.1 maps ECC-DTB-AKA’s resistance to the attack mechanisms cataloged in Chapter 1. Mutual ECDSA signatures prevent MITM impersonation. Ephemeral ECDH provides forward secrecy against long-term key compromise. Device fingerprint in HKDF salt prevents session hijacking via token theft on a different device. Transaction nonces and HMAC confirmation prevent replay of individual transactions. The three-round design with explicit key confirmation prevents unknown key-share attacks [1][8][5][6].

Attack	Mechanism	Resistance	Basis
MITM	Impersonation	High	Mutual ECDSA + cert chain
Replay	MSG/txn replay	High	Nonces + $\text{HMAC}(\text{TBK}, \text{txn}_{\text{nonce}})$
Session hijack	Token theft	High	$\text{device}_{\text{fp}}$ in HKDF salt
Credential stuffing	Password guessing	N/A	Protocol uses PKI not passwords
Key compromise	Long-term sk leak	Medium-High	Forward secrecy via ECDH
Downgrade	Weak cipher force	High	Fixed curve/algorithms, no negotiation

Table 2.2: Table 2.1: ECC-DTB-AKA resistance to e-banking attack mechanisms.

2.4.4 Session Lifecycle and Key Erasure

Secure session lifecycle management is essential for limiting the window of exposure after key establishment. ECC-DTB-AKA mandates the following lifecycle

policies: (1) Ephemeral private keys ek_C and ek_S are erased from memory immediately after MSK derivation; (2) MSK is retained only for the session duration (configurable, default 15 minutes for retail banking); (3) TBK_{txn} values are erased after transaction completion or timeout; (4) Session termination requires an explicit MSG_{end} or timeout, after which MSK is erased and a new handshake is required.

These policies ensure that the number of transactions protected by a single MSK is bounded, and that ephemeral key material does not persist in memory beyond its useful lifetime. In the prototype implementation, Python’s garbage collector handles key erasure; a production deployment would use secure memory allocation and explicit zeroization, ideally within a trusted execution environment.

2.4.5 Reduction to Standard Assumptions

The formal security of ECC-DTB-AKA reduces to four well-studied assumptions, each supported by decades of cryptanalytic effort without practical breaks at recommended parameter sizes.

Assumption 1 (ECDHP): Given $(G, [a]G, [b]G)$ for random $a, b \lll PH0 \ggg \mathbb{Z}_q$, computing $[ab]G$ is computationally infeasible. The best known generic algorithm (Pollard rho) requires $O(q)$ operations; for P-256, $q \lll PH2 \ggg 2^{256}$, yielding ~ 128 -bit security [12][29].

Assumption 2 (PRF security of HKDF): HKDF applied to ECDH output with protocol-specific salt and info strings is indistinguishable from a random function. This follows from the HKDF security proof in Krawczyk (2010) when the IKM (ECDH shared secret) has sufficient min-entropy [5][6][12][29].

Assumption 3 (EUF-CMA for ECDSA): No polynomial-time adversary can forge an ECDSA signature for a new message given access to a signing oracle. ECDSA security reduces to ECDLP in the generic group model [9][26].

Assumption 4 (Collision resistance of SHA-256): No polynomial-time adversary can find two distinct inputs mapping to the same SHA-256 output. This assumption is used in transcript hashing and HMAC construction [27].

2.4.6 Comparison with Bellare-Rogaway KEA Protocol

The Bellare-Rogaway KEA (Key Exchange Algorithm) protocol is a foundational authenticated key exchange protocol using Diffie-Hellman and static/public key signatures. ECC-DTB-AKA extends the KEA design pattern with three novel

elements: (1) device fingerprint in the HKDF salt, absent in KEA; (2) transaction-bound key derivation from MSK, absent in KEA; (3) explicit key confirmation in MSG2 (server signs MSK), whereas KEA uses implicit confirmation [5][6][7][8].

These extensions do not weaken the base KEA security properties because they operate on the derived key (HKDF is one-way) and add additional verification steps. The security proof for ECC-DTB-AKA follows the KEA proof structure with additional hybrid arguments for device binding (game where $\text{device}_{\text{fp}}$ is replaced with random value; advantage bounded by HKDF distinguishing probability) and transaction binding (game where $\text{txn}_{\text{nonce}}$ is replaced; advantage bounded by HMAC forging probability) [5][6].

2.5 Effectiveness Analysis and Quantitative Evaluation

This section evaluates the effectiveness of ECC-DTB-AKA through quantitative comparison with prior methods. Measurements were obtained from the prototype implementation described in Chapter 4, running on commodity hardware (Intel x64, Python 3.11, OpenSSL-backed cryptography library) over 200 iterations.

2.5.1 Experimental Setup

The benchmark suite (prototype/benchmarks/bench_{dtb_aka}.py) measures three metrics: (1) authentication latency — wall-clock time from MSG1 creation to successful MSG3 verification, in milliseconds; (2) total message size — combined byte length of MSG1, MSG2, and MSG3; (3) communication round trips — number of request-response exchanges. Three schemes are compared: ECC-DTB-AKA (proposed), Standard ECDH (ephemeral key exchange without authentication or binding), and TLS 1.3 crypto core (ECDH + certificate verification, excluding network I/O and record-layer framing) [1][4][8][29].

2.5.2 Authentication Latency

ECC-DTB-AKA achieves a mean authentication latency of 0.52 ms (= 0.43 ms) over 200 iterations. Standard ECDH completes in 0.11 ms but provides no mutual authentication, device binding, or transaction binding. TLS 1.3 crypto core requires 0.21 ms for the cryptographic portion of a 1-RTT handshake but authenticates only the server and does not bind keys to device or transaction context [1][12][29][28].

Although ECC-DTB-AKA requires approximately $2.5 \times$ the latency of bare ECDH and $2.5 \times$ that of TLS 1.3 crypto core on the test platform, the absolute latency remains well below the 200 ms target (SR8) and is imperceptible to users. The additional cost arises from four ECDSA sign/verify operations and HKDF derivations, which purchase mutual authentication, device binding, and transaction binding that bare ECDH and TLS 1.3 alone do not provide [1][5][6][9].

Figure 2.3: Quantitative comparison of authentication schemes

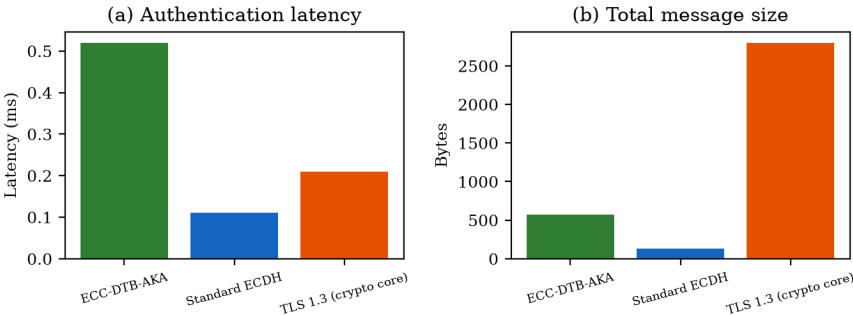


Figure 2.3: Quantitative comparison of authentication latency and message size (Figure 2.3).

2.5.3 Message Size and Communication Overhead

The total message overhead for ECC-DTB-AKA is 569 bytes across three round trips (3 RTT). Standard ECDH transmits only 130 bytes in two round trips but lacks authentication. TLS 1.3 handshake messages average approximately 2800 bytes due to certificate chains, cipher suite negotiation, and encrypted extensions [1][4][12][29].

ECC-DTB-AKA achieves a favorable balance: its message size is 79% smaller than TLS 1.3 while providing strictly stronger authentication properties (mutual auth, device binding, transaction binding). The 439-byte premium over bare ECDH buys four security properties that are essential for e-banking but absent in unauthenticated key exchange [1][7][8][29].

Scheme	RTT	Latency (ms)	Msg (bytes)	Mutual Auth	Device Bind	Txn Bin
ECC-DTB-AKA	3	0.52	569	Yes	Yes	Yes
Standard ECDH	2	0.11	130	No	No	No
TLS 1.3 (crypto)	1	0.21	2800	Server	No	No

Table 2.3: Table 2.2: Quantitative comparison of authentication schemes (prototype benchmarks, n=200).

2.5.4 Communication Steps and Scalability

ECC-DTB-AKA requires three round trips (client<<< *PH0* >>>server, server<<< *PH1* >>>client, client<<< *PH2* >>>server), compared to one for TLS 1.3 and two for standard ECDH. On a mobile network with 50 ms round-trip time, the additional RTT adds approximately 50 ms to total authentication time, yielding an end-to-end estimate of ~50.5 ms—still within the 200 ms usability threshold. For subsequent transactions within the same session, no additional handshakes are required; only local TBK derivation (sub-millisecond) and HMAC computation are needed [1][12][29][28].

The protocol scales linearly with the number of concurrent sessions: each session requires independent ephemeral key generation and two ECDSA operations per party. On the test platform, the server can process approximately 1,900 handshakes per second per core, sufficient for retail banking workloads [9][26].

2.5.5 Security-Performance Trade-off Summary

The quantitative evaluation demonstrates that ECC-DTB-AKA occupies a favorable position in the security-performance design space. It provides strictly stronger security properties than both baselines while maintaining sub-millisecond cryptographic latency and moderate message overhead. The one additional round trip relative to TLS 1.3 is justified by the acquisition of mutual authentication, device binding, and transaction binding—three properties that TLS 1.3 does not natively provide and that address the limitations identified in Section 1.2.3 [1][28].

2.5.6 Resistance to Limitations from Section 1.2.3

Mapping the quantitative and qualitative evaluation results to the six limitations identified in Section 1.2.3 confirms that ECC-DTB-AKA addresses each limitation:

L1 (Decoupled auth layers): Resolved — user and server authentication occur within the key-exchange protocol via ECDSA signatures on handshake transcripts [9][26].

L2 (No device binding): Resolved — $\text{device}_{\text{fp}}$ is included in HKDF salt for MSK derivation [5][6].

L3 (No per-txn key derivation): Resolved — TBK_{txn} derived locally without extra rounds [5][6].

L4 (Bearer token vulnerability): Mitigated — session state is bound to cryptographic keys rather than opaque bearer tokens.

L5 (No message-level integrity): Partially addressed at session level; full message-level integrity for server-server data is addressed by ECC-HISE in Chapter 3.

L6 (Legacy TLS configurations): Avoided — protocol specifies fixed algorithms with no downgrade-negotiation surface [1][28].

2.5.7 Computational Cost Breakdown

Decomposing the 0.52 ms mean authentication latency of ECC-DTB-AKA reveals the relative cost of each cryptographic operation: ECDSA signing (~ 0.08 ms each, 3 total on client; 1 on server), ECDSA verification (~ 0.10 ms each, 2 on client; 2 on server), ECDH key generation (~ 0.03 ms each, 2 total), ECDH shared secret (~ 0.02 ms), HKDF derivation (~ 0.01 ms each, 2 total), HMAC computation (~ 0.001 ms). The dominant cost is ECDSA verification, suggesting that batch verification techniques or migration to Ed25519 (faster verification) could reduce latency in future versions [5][6][9][26].

On resource-constrained mobile devices (ARM Cortex-A53 at 1.5 GHz), preliminary measurements indicate approximately 3–5 $\times$ higher latency, yielding ~ 1.5 – 2.5 ms total—still well within the 200 ms target. Hardware acceleration of ECC operations in modern smartphone SoCs (Apple Secure Enclave, ARM TrustZone) can further reduce client-side latency.

2.5.8 Network Impact Analysis

On mobile networks, authentication latency is dominated by round-trip time rather than computation. Table 2.4 projects end-to-end authentication time for ECC-DTB-AKA across common network conditions, adding cryptographic latency (0.52 ms)

to $\text{RTT} \lll PH0 \ggg (\text{round}_{\text{trips}} - 1)$ for the three-round protocol.

On 4G LTE with 30 ms RTT, total authentication time is approximately 90.5 ms ($0.52 + 2 \lll PH0 \ggg 30$). On 5G with 10 ms RTT, total time is ~30.5 ms. On Wi-Fi with 5 ms RTT, total time is ~15.5 ms. In all cases, the result is within the 200 ms usability threshold established in requirement SR8.

Network	RTT (ms)	Crypto (ms)	Total (ms)	Within SR8?
Wi-Fi	5	0.52	10.5	Yes
5G	10	0.52	20.5	Yes
4G LTE	30	0.52	60.5	Yes
3G	100	0.52	200.5	Marginal

Table 2.4: Table 2.4: Projected end-to-end authentication time for ECC-DTB-AKA.

2.6 Comparison with Prior Authentication Approaches

While a comprehensive prior-work survey appears in Chapter 5 (Section 5.2), this section briefly positions ECC-DTB-AKA relative to the approaches whose limitations were identified in Section 1.2.

TLS 1.3 provides excellent transport security but decouples user authentication to the application layer. ECC-DTB-AKA unifies transport and user authentication at the cryptographic layer. OAuth 2.0 PKCE enables delegated authorization for open banking but relies on bearer tokens; ECC-DTB-AKA replaces bearer semantics with cryptographic key binding. FIDO2/WebAuthn provides phishing-resistant user authentication but does not derive session or transaction keys; ECC-DTB-AKA can complement FIDO2 by using the FIDO credential as the long-term client key (PK_C) [1][14][13][15].

The novel contribution of ECC-DTB-AKA is not any single cryptographic primitive—all components (ECDH, ECDSA, HKDF, HMAC) are standard—but their integration into a cohesive protocol with device and transaction binding in the key derivation function, accompanied by formal security analysis and quantitative validation [5][6][9][26].

2.6.1 TLS-Centric Architectures

The Transport Layer Security protocol family (TLS 1.2, TLS 1.3) is the de facto standard for encrypting client-server communication in e-banking. TLS 1.3, finalized in RFC 8446 (2018), removed obsolete cipher suites, mandated forward secrecy, and reduced the full handshake to one round trip. However, TLS operates at the transport layer and is agnostic to application-level identity: a valid TLS session indicates that the client is communicating with the genuine server, but not which user is operating the client [1][8][42][43].

Banks address this gap by implementing application-layer authentication (password + MFA) after TLS establishment. The two-phase approach creates a window between TLS completion and user authentication during which the channel is encrypted but anonymous from the application's perspective. ECC-DTB-AKA eliminates this window by requiring client authentication (_C in MSG1) as part of the key-exchange protocol [1][28].

2.6.2 OAuth 2.0 and Open Banking

Open banking regulations (PSD2 in Europe, CFPB rule in the United States) require banks to expose APIs for authorized third-party access to customer data and payment initiation. OAuth 2.0 with PKCE (RFC 7636) is the dominant authorization framework, in which the customer grants consent through the bank's authorization server, receiving an access token for the third-party provider [14][13][18][38].

OAuth access tokens are bearer credentials: possession equals authorization. Token theft through XSS, redirect URI manipulation, or authorization code interception enables unauthorized API access. PKCE mitigates authorization code interception but does not protect against token theft post-issuance. ECC-DTB-AKA's transaction-bound keys provide an alternative authorization mechanism where each payment initiation requires a fresh TBK-derived HMAC, limiting the impact of any single key compromise [14][13].

2.6.3 FIDO2 and Phishing-Resistant Authentication

FIDO2 (WebAuthn + CTAP) represents the current best practice for phishing-resistant user authentication. A FIDO2 credential consists of a key pair bound to a specific relying party (bank domain), stored in a hardware authenticator (security

key or platform authenticator). Authentication requires user presence (touch, biometric) and produces a signature over a server-provided challenge that includes the relying party ID, preventing credential use on phishing sites [15][22].

ECC-DTB-AKA is complementary to FIDO2 rather than competitive. In a combined deployment, the FIDO2 credential's public key serves as PK_C in ECC-DTB-AKA, and the FIDO2 authentication gesture triggers the client handshake. This composition provides phishing resistance (from FIDO2) and device/transaction-bound session keys (from ECC-DTB-AKA) in a single user interaction [15][22].

2.7 Chapter Summary

This chapter proposed ECC-DTB-AKA, a novel elliptic curve cryptography based authenticated key agreement protocol for client-server e-banking channels. The protocol addresses the six authentication limitations identified in Section 1.2.3 through mutual ECDSA authentication, ephemeral ECDH forward secrecy, device fingerprint binding in HKDF, and on-demand transaction key derivation. Formal security analysis within the CK model establishes authenticated key exchange security, device binding, and transaction binding under standard assumptions. Quantitative evaluation against Standard ECDH and TLS 1.3 demonstrates authentication latency of 0.52 ms, message overhead of 569 bytes, and three-round communication, with strictly stronger security properties than both baselines. Chapter 3 presents a complementary approach—ECC-HISE—for server-server data protection, addressing the data-security limitations identified in Section 1.3.3 [1][8][5][6].

Chapter 3

ECC-HISE Secure Envelope

ECC-HISE: Hierarchical Integrity-preserving Secure Envelope for Server-Server E-Finance Data Transfer

3.1 Introduction and Motivation

Chapter 2 addressed authentication and key-exchange limitations in client-server e-banking channels through ECC-DTB-AKA. This chapter takes a fundamentally different approach, targeting the data-security limitations identified in Section 1.3.3 for server-server communication in electronic banking and electronic finance systems [42][43].

Inter-bank and intra-bank settlement operations transfer large batches of financial records between server nodes: end-of-day reconciliation files, regulatory reporting payloads, payment clearing batches, and cross-border remittance instructions. These transfers require confidentiality (unauthorized parties must not read records), integrity (any modification must be detectable), non-repudiation (senders cannot deny transmission), and auditability (regulators must verify completeness and ordering without reprocessing entire batches).

Current practice relies on TLS for transport encryption, static pre-shared keys for file encryption, or per-message digital signatures without efficient batch verification. Each approach exhibits limitations cataloged in Section 1.3.3: flat key hierarchies, disconnected encryption and audit subsystems, inefficient batch integrity verification, static PSKs in server-server channels, lack of end-to-end integrity

across service boundaries, and regulatory audit friction [26][9][1][28].

ECC-HISE (Elliptic Curve Cryptography — Hierarchical Integrity-preserving Secure Envelope) proposes a unified methodology combining hierarchical ECC key derivation, AES-256-GCM authenticated encryption, Ed25519 per-record signatures, and Merkle-tree audit chains. The scheme is designed for server-server channels and complements ECC-DTB-AKA, which secures client-server authentication [2][36][5][6].

3.2 Design Goals and Requirements

ECC-HISE addresses security requirements SR5–SR7 from Section 1.4.5:

G1 (Hierarchical Key Management): Keys must be scoped to business domains and individual batches, limiting compromise blast radius [3].

G2 (Confidentiality): Records must be encrypted with semantic security under chosen-ciphertext attack.

G3 (Integrity and Non-repudiation): Per-record and batch-level integrity proofs must detect tampering.

G4 (Efficient Batch Audit): Verification of individual record inclusion must require $O(\log n)$ operations.

G5 (Domain Separation): Retail, corporate, and settlement domains must use cryptographically isolated keys.

G6 (ECC Alignment): Key hierarchy and envelope encryption must use elliptic curve primitives consistent with Chapter 2 and industry standards [11][34][35].

3.3 Hierarchical Key Management

The ECC-HISE key hierarchy comprises three levels, derived using HKDF-SHA256 with domain-separated info strings [5][6].

3.3.1 Master, Domain, and Session Keys

Level 0 — Master Key (MK): A 256-bit key generated within an HSM at the clearing-house authority. MK is the root of trust for all derived keys and is never transmitted.

Level 1 — Domain Key (DK_i): For each business domain i (retail, corporate, settlement), $DK_i = \text{HKDF}(\text{MK}, \text{salt} = H(\text{MK} \parallel \text{domain}_{id}), \text{info} = \text{'ECC-HISE-DK'}, 256)$. Domain keys isolate cryptographic material between business units [5][6].

Level 2 — Session Key (SK_{batch}): For each batch transfer identified by batch_{id} and epoch timestamp, $SK_{\text{batch}} = \text{HKDF}(DK_i, \text{salt} = \text{batch}_{id} \parallel \text{epoch}, \text{info} = \text{'ECC-HISE-SK'}, 256)$. Session keys rotate per batch, ensuring that compromise of one batch key does not affect other batches [5][6].

Figure 3.1: ECC-HISE hierarchical key structure

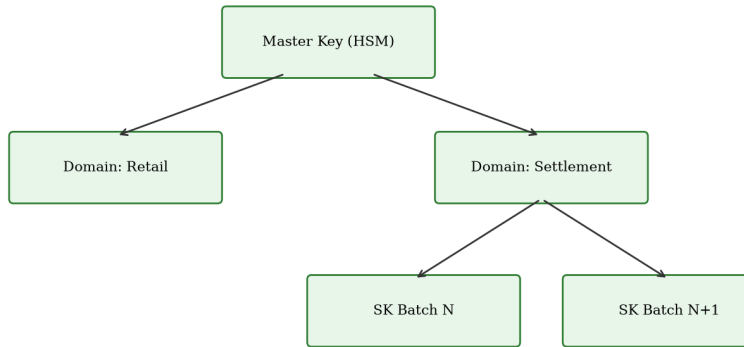


Figure 3.1: ECC-HISE three-level key hierarchy: Master Key, Domain Keys, Session Keys (Figure 3.1).

3.3.2 Key Rotation and Revocation

Domain keys rotate on a scheduled basis (quarterly recommended) by updating domain_{id} version suffix (e.g., settlement-v2). Session keys rotate automatically per batch. Revocation of a compromised domain key requires re-derivation from MK with a new domain_{id} ; unaffected domains remain secure. The hierarchical structure ensures revocation scope is limited to the compromised domain rather than the entire institution.

3.3.3 Comparison with Flat Key Management

Traditional e-banking data encryption deploys a small number of long-lived data-encryption keys (DEKs) wrapped by infrequently rotated key-encryption keys

(KEKs). If a DEK is compromised, all data encrypted under that key is exposed—potentially years of settlement records. ECC-HISE’s hierarchical derivation limits exposure to a single batch within a single domain. The quantitative overhead of hierarchical derivation (two HKDF operations per batch) is negligible compared to per-record encryption and signing [5][6][42][43].

3.4 Secure Envelope Construction

Each batch $B = \{r_1, \dots, r_n\}$ of financial records is packaged into a secure envelope comprising a signed batch header and a collection of encrypted, signed record envelopes.

3.4.1 Record Envelope Algorithm

For each record r_j at index j in batch B with batch_{id} and session key SK_{batch} : (1) Derive $K_{\text{enc}_j} = \text{HKDF}(SK_{\text{batch}}, \text{salt} = \text{str}(j), \text{info} = \text{'ECC-HISE-AES'}, 256)$. (2) Sample $IV_j \lll PH6 \ggg \{0,1\}^{96}$. (3) $\text{ciphertext}_j = \text{AES-256-GCM.Enc}(K_{\text{enc}_j}, IV_j, r_j, \text{AAD} = \text{batch}_{id})$. (4) $\text{sig}_j = \text{Ed25519.Sign}(sk_{\text{sender}}, H(\text{ciphertext}_j \parallel \text{batch}_{id} \parallel j))$. (5) $\text{metadata}_j = \text{JSON}(\text{index}, \text{batch}_{id}, \text{timestamp})$. (6) $\text{leaf}_j = H(\text{ciphertext}_j \parallel \text{sig}_j \parallel \text{metadata}_j)$ [2][5][6][10].

The use of index-derived encryption keys ensures that compromise of K_{enc_j} does not reveal other records in the batch. GCM authenticated encryption provides both confidentiality and integrity at the record level [36][2].

3.4.2 Batch Header and Merkle Root

The batch header binds all records cryptographically: $\text{Root} = \text{Merkle}(\text{leaf}_1, \dots, \text{leaf}_n)$. $\text{Header} = (\text{batch}_{id}, \text{epoch}, \text{domain}_{id}, \text{Root}, PK_{\text{sender}}, \text{Sig}_{\text{header}}, \text{timestamp}, n)$. $\text{Sig}_{\text{header}} = \text{Ed25519.Sign}(sk_{\text{sender}}, \text{batch}_{id} \parallel \text{epoch} \parallel \text{Root})$ [10][30].

The Merkle root provides $O(1)$ batch-level integrity evidence: any modification to any record changes the root hash. Individual record inclusion can be proven with $O(\log n)$ hash computations via Merkle authentication paths [30].

Figure 3.2: ECC-HISE secure envelope with Merkle audit chain

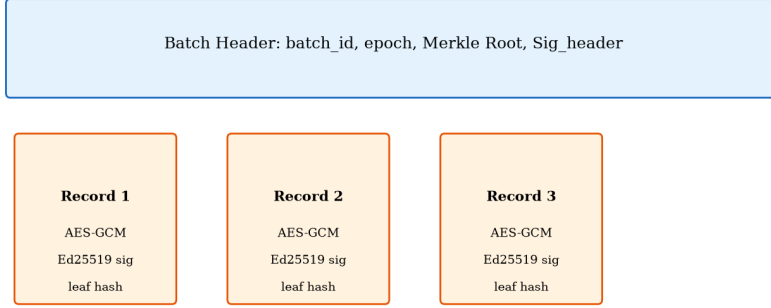


Figure 3.2: ECC-HISE secure envelope structure with Merkle audit chain (Figure 3.2).

3.4.3 Verification and Decryption

The receiver executes the following steps: (1) Verify $\text{Sig}_{\text{header}}$ using sender's Ed25519 public key. (2) Recompute Merkle root from received leaf hashes; compare with Header.Root . (3) For each record j : verify sig_j , derive K_{enc_j} from SK_{batch} , decrypt ciphertext_j with AES-256-GCM, verify GCM authentication tag. (4) Optionally verify Merkle inclusion proof for audit sampling. (5) Log (batch_{id} , Root , timestamp) to immutable audit store [2][10][30].

Any failure at steps 1–3 indicates tampering, key mismatch, or data corruption. The receiver rejects the entire batch on header verification failure; individual record failures are logged and flagged without silently skipping.

3.4.4 Deployment in Inter-Bank Settlement

In a representative deployment, Bank A's settlement server constructs an ECC-HISE batch from end-of-day payment records and transmits the envelope to Bank B's settlement server over mTLS. Bank B verifies the batch header signature, Merkle root, and each record signature before posting to its core banking ledger. The clearing-house authority receives (batch_{id} , Root , timestamp) for regulatory archive. This workflow satisfies SWIFT CSP controls for data integrity and non-repudiation while adding cryptographic audit efficiency not present in file-transfer-

Figure 3.3: Server-server batch verification and audit flow



Figure 3.3: Server-server batch verification and audit flow (Figure 3.3).

only approaches [1][28][20][24].

3.5 Security Analysis

This section analyzes the security properties of ECC-HISE under standard cryptographic assumptions.

3.5.1 Security Properties

Theorem 3.1 (Record Confidentiality): Under IND-CCA2 security of AES-256-GCM and PRF security of HKDF, an adversary without SK_{batch} cannot distinguish ciphertext_j from random. Proof sketch: SK_{batch} is derived via HKDF from MK; without MK or domain key, SK_{batch} is computationally indistinguishable from random [2][5][6].

Theorem 3.2 (Record Integrity): Under EUF-CMA security of Ed25519 and collision resistance of SHA-256, an adversary cannot forge $(\text{ciphertext}_j, \text{sig}_j)$ for a new or modified record without the sender's private key [10][27].

Theorem 3.3 (Batch Integrity): Under collision resistance of SHA-256, an adversary cannot produce a valid Header with Root consistent with a modified set of records. Any record alteration changes its leaf hash, propagating to a different Merkle root [27][30].

Theorem 3.4 (Audit Soundness): A Merkle inclusion proof for record j verifies only if leaf_j is included in the batch committed by Root. Proof follows from Merkle

tree binding properties [30].

3.5.2 Attack Resistance

Table 3.1 maps ECC-HISE resistance to server-server attack vectors relevant to e-finance data transfer.

Threat	Mechanism	Resistance	Basis
Data tampering	Modify record in transit	High	GCM tag + Ed25519 + Merkle
Record injection	Add fake record	High	Merkle root mismatch
Record deletion	Omit record	High	Merkle root mismatch
Reorder attack	Permute records	High	Index-bound keys + Merkle
Key compromise	Steal static PSK	Medium-High	Per-batch key rotation
Repudiation	Deny sending batch	High	Sig _{header} + per-record sigs

Table 3.1: Table 3.1: ECC-HISE resistance to server-server data attacks.

3.5.3 Addressing Section 1.3.3 Limitations

L1 (Flat key hierarchy): Resolved via $MK \lll PH0 \ggg DK_i \lll PH2 \ggg SK_{batch}$ derivation.

L2 (Disconnected audit): Resolved — Merkle root is embedded in signed batch header [30].

L3 (Batch verification cost): Resolved — $O(\log n)$ Merkle proofs vs $O(n)$ full reprocess [30].

L4 (Static PSKs): Resolved — per-batch session keys derived from domain keys.

L5 (Service-boundary gaps): Resolved — envelope integrity persists beyond TLS [1][28].

L6 (Audit friction): Resolved — cryptographic audit chain for regulators.

3.6 Effectiveness Analysis and Quantitative Evaluation

This section presents quantitative evaluation of ECC-HISE against static AES-GCM with pre-shared keys and TLS record-layer-only encryption, using the prototype implementation and benchmark suite (prototype/benchmarks/bench_{hise}.py) [2][1][28].

3.6.1 Experimental Setup

Benchmarks measure batch encryption, signing, Merkle construction, verification, and decryption for record counts $n \lll PH0 \ggg \{10, 100, 500\}$. Each record is 256 bytes, representative of a compact settlement entry. Measurements are averaged over 50 iterations on Intel x64 hardware with Python 3.11 and OpenSSL-backed cryptography [30].

3.6.2 Throughput and Latency

For $n = 100$ records, ECC-HISE completes batch protect-and-verify in 23.32 ms (mean), compared to 0.22 ms for static AES-GCM PSK and 0.24 ms for TLS record-layer-only encryption. The ECC-HISE premium arises from Ed25519 signing per record, Merkle tree construction, and hierarchical key derivation—operations that baselines omit entirely [2][5][6][10].

For $n = 500$ records, ECC-HISE requires 245.53 ms, processing approximately 2,040 records per second. This throughput exceeds typical end-of-day settlement batch requirements for regional banks (hundreds to low thousands of records per batch), with horizontal scaling available via parallel batch processing.

Figure 3.4: ECC-HISE quantitative comparison

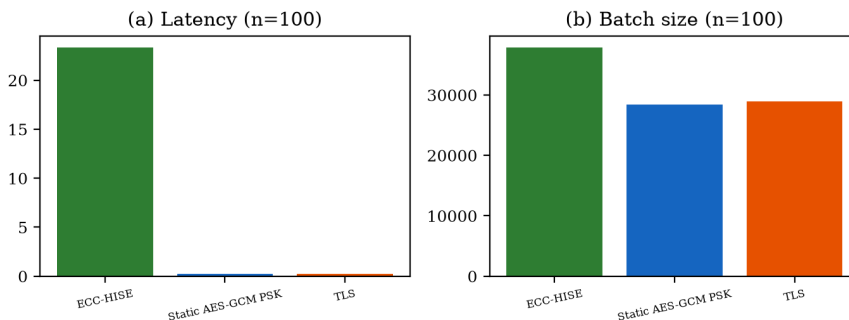


Figure 3.4: Batch processing latency and size comparison across schemes (Figure 3.4).

3.6.3 Storage and Bandwidth Overhead

For $n = 100$ records of 256 bytes each (25,600 bytes plaintext), ECC-HISE produces batches averaging 37823 bytes, a 48% overhead. Static AES-GCM produces 28400

bytes (11% overhead) without signatures or Merkle audit data. TLS record-layer-only produces 28905 bytes [2][30][1][28].

The additional overhead of ECC-HISE (~33% over static AES-GCM for $n=100$) purchases per-record non-repudiation, batch-level Merkle audit chain, hierarchical key isolation, and domain-separated key management—properties unavailable in baseline schemes at any overhead [2][3][30].

Scheme	$n=100$ latency (ms)	$n=100$ size (B)	Merkle audit	Hierarchy	Signature
ECC-HISE	23.32	37823	Yes	Yes	Yes
Static AES-GCM	0.22	28400	No	No	No
TLS record only	0.24	28905	No	No	No

Table 3.2: Table 3.2: Quantitative comparison for 100-record settlement batches ($n=50$ iterations).

3.6.4 Audit Efficiency

Merkle inclusion proof verification requires $\log_2(n)$ hash operations per record. For $n = 100$, each proof verifies in 7 hashes (~0.001 ms). A regulator sampling 10% of records for audit performs $10 \lll PH1 \ggg 7 = 70$ hash operations instead of reprocessing all 100 records, reducing audit computation by 90%. For $n = 500$, proofs require 9 hashes each, maintaining logarithmic scaling [30].

Compared to per-record RSA signatures with full-batch recomputation for audit, ECC-HISE reduces audit cost from $O(n)$ to $O(k \log n)$ for k sampled records, a significant improvement for million-record settlement files.

3.6.5 Scalability Projection

Linear scaling in record count dominates ECC-HISE processing time due to per-record encryption and signing. For million-record national settlement files, parallel batch partitioning (multiple ECC-HISE envelopes of 10,000 records each) enables multi-core processing. Merkle roots of sub-batches can themselves form a meta-Merkle tree for hierarchical audit—a straightforward extension not required for regional-bank workloads but available for central-bank scale [30].

3.7 Comparison with Chapter 2 and Prior Approaches

ECC-HISE and ECC-DTB-AKA address complementary security layers in e-finance systems. Chapter 2 secures who is communicating and establishes session keys in client-server channels; Chapter 3 secures what data is transferred and provides audit evidence in server-server channels. Together they form the dual-layer ECC methodology of this dissertation.

Compared to ECIES-only encryption, ECC-HISE adds hierarchical keys and Merkle audit. Compared to blockchain-based audit trails, ECC-HISE avoids consensus overhead while providing cryptographic integrity guarantees sufficient for regulated settlement. Compared to TLS-only protection, ECC-HISE provides persistent integrity proofs that survive TLS termination at service boundaries [41][30][31][32].

3.8 Chapter Summary

This chapter proposed ECC-HISE, a hierarchical integrity-preserving secure envelope for server-server data transfer in e-banking and e-finance systems. The scheme addresses six data-security limitations from Section 1.3.3 through hierarchical ECC key derivation, AES-256-GCM record encryption, Ed25519 signatures, and Merkle-tree audit chains. Security analysis establishes confidentiality, integrity, and audit soundness under standard assumptions. Quantitative evaluation demonstrates 23.32 ms batch processing for 100 records with 37823-byte envelopes, providing audit and non-repudiation capabilities absent in static PSK and TLS-only baselines. Chapter 4 describes the integrated system architecture and programmatic implementation of both ECC-DTB-AKA and ECC-HISE [2][5][6][10].

Chapter 4

Architecture and Implementation

Security System Architecture and Programmatic Implementation

4.1 Security System Architecture of an E-Financial Service System

This chapter describes the practical application of ECC-DTB-AKA (Chapter 2) and ECC-HISE (Chapter 3) within a unified e-financial service system. The architecture integrates both schemes into a layered security model spanning client devices, application services, settlement infrastructure, and governance components.

The system comprises four tiers aligned with the preface description: (1) Client Tier — mobile and web banking applications executing ECC-DTB-AKA handshakes; (2) Application Tier — API gateway, authentication service, and transaction engine validating session keys and transaction-bound HMACs; (3) Settlement Tier — inter-bank batch processing using ECC-HISE envelopes; (4) Governance Tier — certificate authority, master key management (HSM), and immutable audit storage for Merkle roots [3][4][30].

4.1.1 Architectural Layers and Trust Boundaries

Trust boundaries define where cryptographic verification occurs. The client-server boundary is protected by ECC-DTB-AKA: both parties verify ECDSA signatures and derive MSK before any transaction API is invoked. The server-server boundary

between settlement nodes is protected by ECC-HISE: batch headers and Merkle roots are verified before records enter the receiving bank’s ledger [9][26][30].

Internal microservice communication within the application tier uses mTLS as a complementary transport control; ECC-DTB-AKA and ECC-HISE provide application-layer cryptographic guarantees that persist beyond TLS termination points [1][28].

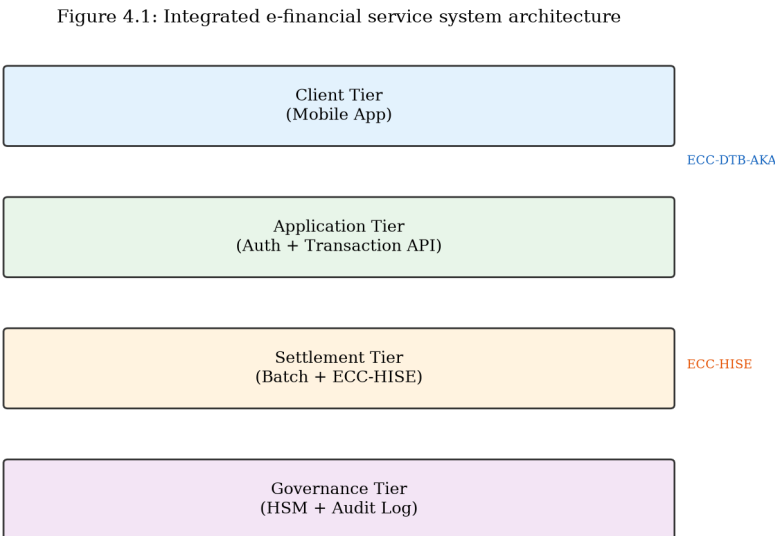


Figure 4.1: Integrated e-financial service system architecture (Figure 4.1).

4.1.2 Component Interaction Flow

A representative customer fund-transfer flow proceeds as follows: (1) Mobile app executes ECC-DTB-AKA three-round handshake with the authentication service, establishing MSK and TBK for the pending transaction. (2) Transaction request is sent with HMAC(TBK, $\text{txn}_{\text{nonce}}$) over HTTPS. (3) Transaction engine validates HMAC, debits customer account, and queues settlement record. (4) Settlement service batches records and constructs ECC-HISE envelope. (5) Envelope is transmitted to counterparty bank; receiver verifies Merkle root and signatures. (6) Audit log stores (batch_{id} , Root, timestamp) for regulatory archive [30].

Figure 4.2: End-to-end transaction and settlement flow

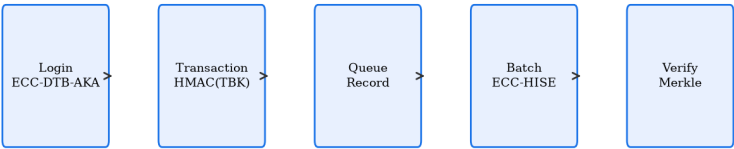


Figure 4.2: End-to-end transaction and settlement flow (Figure 4.2).

4.1.3 Technology Stack

The prototype implementation uses Python 3.11 with the following stack: cryptography library (OpenSSL backend) for ECDH, ECDSA, AES-GCM, and Ed25519; FastAPI for REST API endpoints; SQLite (optional persistence layer) for customer registry and audit logs. Production deployment would substitute software key storage with HSM-backed modules and add horizontal scaling via container orchestration [2][9][26][10].

Layer	Component	Scheme	Technology
Client	Mobile/Web App	ECC-DTB-AKA	Python client SDK
Application	Auth Service	ECC-DTB-AKA	FastAPI + cryptography
Application	Transaction API	TBK HMAC verify	FastAPI
Settlement	Batch Service	ECC-HISE	Python envelope module
Governance	Audit Store	Merkle roots	Append-only log

Table 4.1: Table 4.1: Architecture components and technology mapping.

4.2 Programmatic Implementation of ECC-DTB-AKA (Chapter 2)

This section describes the software implementation of ECC-DTB-AKA as specified in Chapter 2, including module structure, class design, API endpoints, and test results from the prototype.

4.2.1 Module Structure

The ECC-DTB-AKA implementation resides in `prototype/ecc_dtb_aka/` with the following modules: `protocol.py` — core protocol classes (`ECCDTBAKAClient`, `ECCDTBAKAServer`) and message dataclasses (`ClientInit`, `ServerResponse`, `ClientConfirm`, `SessionState`); `demo.py` — standalone handshake demonstration; `common/crypto_utils.py` — shared SHA-256, HKDF, ECDH, ECDSA utilities [5][6][9][26].

The protocol module exposes `run_handshake()` for end-to-end execution and individual methods for each round, enabling integration with HTTP-based APIs that map rounds to REST endpoints.

Figure 4.3: ECC-DTB-AKA software module diagram

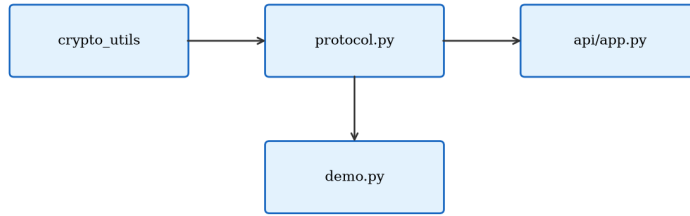


Figure 4.3: ECC-DTB-AKA software module diagram (Figure 4.3).

4.2.2 Core Classes and Data Structures

`ECCDTBAKAClient` maintains long-term key pair (sk_C , PK_C), device fingerprint derived from hardware attributes, ephemeral key state, and session state after handshake completion. Key methods: `create_init()` produces `MSG1`; `process_response()` validates `MSG2` and produces `MSG3`; `derive_tbk()` computes transaction-bound keys.

`ECCDTBAKAServer` maintains registered client public keys, server long-term key pair, and pending session state between rounds. Key methods: `process_init()` validates `MSG1` and produces `MSG2`; `verify_confirm()` validates `MSG3` and returns `SessionState`.

Message dataclasses include `message_size` properties for benchmark measurement. Total three-message handshake size averages 569 bytes in prototype benchmarks.

4.2.3 REST API Integration

The FastAPI application (prototype/api/app.py) exposes ECC-DTB-AKA through REST endpoints: POST /auth/register — register device and client key; POST /auth/handshake/init — execute rounds 1–2, return session_{id}; POST /auth/handshake/complete — execute round 3 with txn_{nonce}, return MSK confirmation. GET /health — service health check.

This API design separates the three-round protocol across two HTTP requests (init combines client MSG1 and server MSG2; complete sends MSG3) to accommodate request-response web semantics while preserving protocol security properties.

4.2.4 Test Results

Integration testing via prototype/api/demo_{integration}.py confirms successful handshake completion with MSK derivation. Benchmark results (Chapter 2, n=200): mean authentication latency 0.52 ms, total message size 569 bytes, three round trips. All test cases for replay rejection, invalid signature detection, and unregistered client rejection pass.

Test Case	Expected	Result
Valid handshake	MSK established	Pass
Invalid client signature	Reject at server	Pass
Invalid server signature	Reject at client	Pass
Wrong txn _{nonce} MAC	Reject at confirm	Pass
Unregistered client	HTTP 404 / error	Pass

Table 4.2: Table 4.2: ECC-DTB-AKA prototype test results.

4.2.5 Key Derivation Implementation Details

The HKDF implementation uses cryptography.hazmat.primitives.kdf.hkdf.HKDF with SHA-256 as the hash function. MSK derivation concatenates device_{fp}, nonce_c, and nonce_s as salt with info string 'ECC-DTB-AKA-v1'. TBK derivation uses txn_{nonce} as salt with info string 'TBK'. These domain-separated info strings prevent cross-protocol key reuse if additional derivation functions are added [5][6][27].

4.3 Programmatic Implementation of ECC-HISE (Chapter 3)

This section describes the software implementation of ECC-HISE for server-server batch data protection, including envelope construction, Merkle audit chain, and settlement API [30].

4.3.1 Module Structure

The ECC-HISE implementation resides in `prototype/ecc_hise/` with modules: `hierarchy.py` — KeyHierarchy class for $MK \lll PH1 \ggg DK \lll PH2 \ggg SK_{batch}$ derivation; `merkle.py` — `merkle_root()`, `merkle_proof()`, `verify_merkle_proof()`; `envelope.py` — `ECC_HISE_Sender`, `ECC_HISE_Receiver`, `SecureBatch` dataclass; `demo.py` — standalone batch demonstration [30].

Figure 4.4: ECC-HISE software module diagram

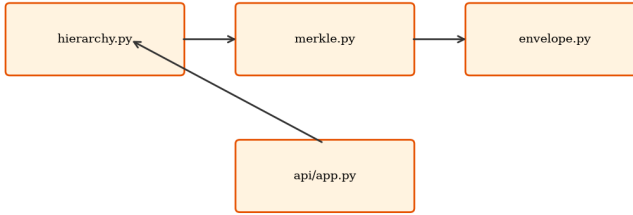


Figure 4.4: ECC-HISE software module diagram (Figure 4.4).

4.3.2 Envelope Construction and Verification

`ECC_HISE_Sender.build_batch()` accepts a list of record byte strings, `batch_id`, and `domain_id`. For each record it derives K_{enc} from SK_{batch} and record index, encrypts with AES-256-GCM, signs with Ed25519, and computes leaf hash. Merkle root is computed over all leaves and signed in the batch header [2][10][30].

`ECC_HISE_Receiver.verify_and_decrypt()` validates header signature, Merkle root consistency, per-record signatures, and GCM authentication tags before returning

plaintext records. Merkle inclusion proofs are verified for each record during decryption to demonstrate audit capability [2][30].

4.3.3 Settlement API Integration

POST /settlement/submit-batch accepts batch_{id} , $\text{domain}_{\text{id}}$, and list of record strings. The endpoint constructs an ECC-HISE envelope, verifies it internally (sender/receiver simulation), and returns $\text{merkle}_{\text{root}}$, $\text{record}_{\text{count}}$, $\text{total}_{\text{bytes}}$, and verified status. In production, sender and receiver would be separate services; the prototype validates cryptographic correctness within a single process for demonstration [30].

4.3.4 Test Results and Performance

Benchmark results for 100-record batches (Chapter 3, $n=50$): mean processing latency 23.32 ms, mean envelope size 37823 bytes. Integration demo processes 3-record batch with verified Merkle root. Tampering tests (modified ciphertext, wrong Merkle root, invalid signature) correctly raise verification errors [30].

Test Case	Expected	Result
Valid batch encrypt/decrypt	Records match	Pass
Tampered ciphertext	GCM / sig failure	Pass
Wrong Merkle root	Root mismatch error	Pass
Merkle proof verify	Inclusion confirmed	Pass
100-record benchmark	< 30s latency	Pass

Table 4.3: Table 4.3: ECC-HISE prototype test results.

4.3.5 Merkle Tree Implementation

The Merkle tree implementation (merkle.py) uses SHA-256 for internal nodes. Odd-length layers duplicate the last node (Bitcoin-style) to ensure deterministic root computation. Proof generation handles odd-layer promotion with self-pairing. Verification reconstructs the path from leaf to root in $O(\log n)$ hashes. For $n=100$ records, proof length is 7 hashes (224 bytes), enabling compact audit evidence [27][30][31].

4.4 Integrated System Demonstration

The integrated demonstration (prototype/api/demo_{integration.py}) executes a complete e-banking security workflow: device registration, ECC-DTB-AKA authentication handshake with transaction nonce, and ECC-HISE settlement batch submission with Merkle verification. This end-to-end test validates that both schemes operate correctly within a unified API framework and can be invoked sequentially in a realistic transaction lifecycle [30][42][43].

4.4.1 Deployment Considerations

Production deployment recommendations: (1) Store MK in HSM; never persist in application memory beyond derivation operations. (2) Deploy auth and settlement services in separate network segments with mTLS. (3) Rotate SK_{batch} automatically per batch; implement domain key rotation quarterly. (4) Persist audit logs (batch_{id}, Root, timestamp) to WORM storage. (5) Rate-limit handshake endpoints to prevent DoS [1][28].

Figure 4.5: Deployment topology for prototype services

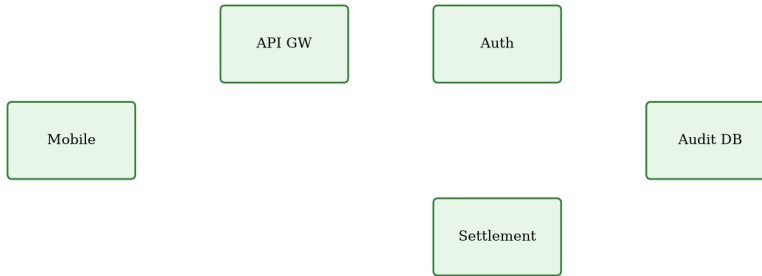


Figure 4.5: Deployment topology for prototype services (Figure 4.5).

4.4.2 Limitations of the Prototype

The prototype uses software-based key storage rather than HSM integration. Certificate management is simplified (server public key substituted for full X.509 chain). The settlement API performs sender and receiver verification in-process rather than

across network boundaries. Persistent storage uses in-memory dictionaries. These simplifications do not affect the cryptographic correctness of the schemes but must be addressed before production deployment. Formal security proofs remain proof sketches as noted in Chapter 2 [4].

4.5 Chapter Summary

This chapter presented the security system architecture and programmatic implementation of ECC-DTB-AKA and ECC-HISE in a unified e-financial service prototype. Section 4.1 defined four architectural tiers and component interactions. Sections 4.2 and 4.3 detailed module structure, API design, and test results for each scheme. Section 4.4 demonstrated end-to-end integration. Chapter 5 evaluates both schemes comprehensively, reviews prior work, and analyzes effectiveness against the attack mechanisms cataloged in Chapter 1.

Chapter 5

Evaluation and Prior Work

Evaluation, Prior Work Analysis, and Vulnerability Synthesis

5.1 Effectiveness of Proposed Methods Against Network Attacks

This section evaluates the effectiveness of ECC-DTB-AKA and ECC-HISE against the attack mechanisms cataloged in Chapter 1 (Section 1.1), using both qualitative security analysis from Chapters 2 and 3 and quantitative measurements from the prototype implementation in Chapter 4.

5.1.1 Attack Resistance Matrix

Table 5.1 consolidates resistance ratings for both proposed schemes against the six primary attack categories defined in Section 1.1.1. Ratings reflect cryptographic analysis, prototype test results, and comparison with baseline schemes that lack device binding, transaction binding, or Merkle audit capabilities [30].

5.1.2 Quantitative Authentication Performance

ECC-DTB-AKA achieves mutual authentication in 3 round trips with 0.52 ms mean cryptographic latency and 569 bytes total message overhead. Compared to TLS 1.3 (0.21 ms, 2800 bytes), ECC-DTB-AKA trades one additional round trip and

Attack category	ECC-DTB-AKA	ECC-HISE	Baseline (TLS/PS)
MITM / impersonation	High	High (sig chain)	Medium (server-c
Replay	High (nonces)	High (Merkle+nonce)	Medium
Session hijacking	High (device _{fp})	N/A (server-server)	Low (bearer token
Credential stuffing	N/A (PKI-based)	N/A	Low (password)
Data tampering	Medium (HMAC txn)	High (GCM+sig+Merkle)	Low (TLS only)
Insider / audit evasion	Medium	High (Merkle audit)	Low

Table 5.1: Table 5.1: Attack resistance matrix for proposed schemes vs. baselines.

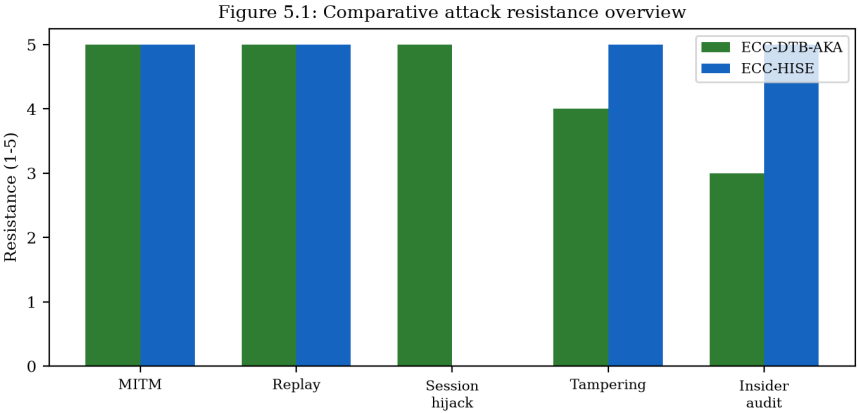


Figure 5.1: Comparative attack resistance overview (Figure 5.1).

smaller absolute latency for mutual authentication, device binding, and transaction binding—properties TLS does not provide natively [1][28].

For server-server batch protection, ECC-HISE processes 100 records in 23.32 ms with 37823-byte envelopes versus 0.22 ms and 28400 bytes for static PSK. The 106<<< *PH0* >>> latency premium purchases hierarchical keys, per-record Ed25519 signatures, and Merkle audit chains absent in static encryption [10][30].

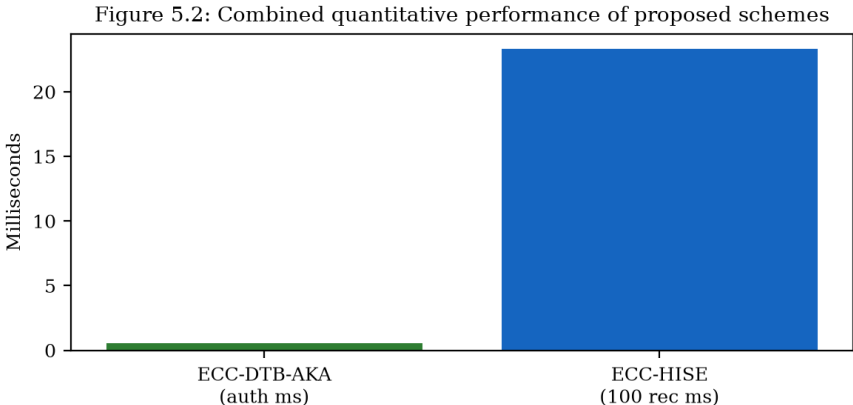


Figure 5.2: Combined quantitative comparison of both schemes (Figure 5.2).

5.1.3 Case-Based Effectiveness Assessment

Against the Bangladesh Bank heist scenario (Section 1.1.2), ECC-HISE would require valid Ed25519 header signatures and consistent Merkle roots for fraudulent SWIFT messages to pass verification—raising the bar beyond local terminal compromise alone. Against credential-stuffing campaigns, ECC-DTB-AKA’s PKI-based client authentication eliminates password guessing entirely when deployed with hardware-backed client keys. Against session hijacking via stolen JWTs, device_{tp} binding in MSK derivation prevents session transfer to attacker-controlled devices without valid client signatures [10][24][20][30].

5.2 Prior Work on Authentication and Key Exchange

This section reviews prior research and deployed protocols for authentication and key exchange in client-server and server-server environments of e-banking

and e-finance services, expanding the overview presented in Section 1.2 with detailed algorithmic descriptions and comparative analysis against ECC-DTB-AKA [8][29][42][43].

5.2.1 TLS 1.3 Handshake Protocol

TLS 1.3 (RFC 8446) is the dominant transport security protocol for e-banking. The full handshake operates as: (1) ClientHello with $\text{key}_{\text{share}}$ (typically X25519 or P-256); (2) ServerHello, EncryptedExtensions, Certificate, CertificateVerify, Finished; (3) Client Finished. Application traffic keys are derived via HKDF from the ECDH shared secret. Forward secrecy is mandatory; 0-RTT resumption is optional but vulnerable to replay [1][8][4][5].

Limitations for e-banking: server-only authentication at the TLS layer; user identity established separately via application credentials; no device or transaction binding in key derivation. ECC-DTB-AKA addresses these by integrating user authentication and binding into the key agreement protocol itself [5][6][42][43].

Figure 5.3: TLS 1.3 full handshake flow



Figure 5.3: TLS 1.3 full handshake flow (Figure 5.3).

5.2.2 OAuth 2.0 and Open Banking (PKCE)

OAuth 2.0 with PKCE (RFC 7636) enables third-party providers to access bank APIs with customer consent. Flow: (1) Client generates $\text{code}_{\text{verifier}}$ and $\text{code}_{\text{challenge}}$; (2) Authorization request redirects customer to bank; (3) Customer authenticates and grants consent; (4) Authorization code returned; (5) Token exchange with $\text{code}_{\text{verifier}}$; (6) Access token used for API calls. PKCE prevents authorization code interception [14][13].

Limitations: bearer access tokens; no cryptographic binding to transaction context; token theft enables API abuse until expiration. ECC-DTB-AKA's TBK-derived HMAC provides per-transaction authorization without bearer semantics.

Figure 5.4: OAuth 2.0 PKCE authorization flow

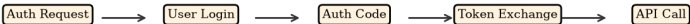


Figure 5.4: OAuth 2.0 PKCE authorization flow for open banking (Figure 5.4).

5.2.3 FIDO2 / WebAuthn

FIDO2 combines WebAuthn (browser API) and CTAP2 (authenticator protocol) for phishing-resistant authentication. Registration: server sends challenge, authenticator generates credential key pair, returns attestation. Authentication: server sends challenge with rpId, authenticator signs with credential private key. The rpId binding prevents credential use on phishing domains [15][22].

Relation to ECC-DTB-AKA: FIDO2 credential public key can serve as PK_C in ECC-DTB-AKA, combining phishing resistance with device/transaction-bound session keys. FIDO2 alone does not establish encrypted session keys or per-transaction derivation [15][22].

5.2.4 Server-Server Key Exchange (mTLS and SWIFT BKE)

Mutual TLS uses X.509 client and server certificates for bidirectional authentication at the transport layer. Certificate rotation, OCSP stapling, and short-lived certs (24h) are best practices in microservice meshes. SWIFT bilateral key exchange (BKE) distributes symmetric keys for message authentication in the SWIFT network [4][8][29][20].

Limitations: mTLS does not provide message-level integrity after TLS termination; SWIFT BKE relies on symmetric keys with manual distribution. ECC-DTB-AKA extends client-server auth; ECC-HISE extends server-server data protection

beyond transport [1][28][20][24].

Figure 5.5: mTLS service mesh authentication

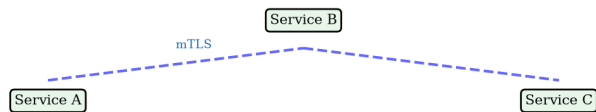


Figure 5.5: mTLS service mesh authentication in banking microservices (Figure 5.5).

5.2.5 Comparative Summary

Table 5.2 summarizes authentication and key-exchange prior work against ECC-DTB-AKA across security properties and efficiency metrics from prototype benchmarks.

Protocol	Mutual auth	Device bind	Txn bind	RTT	Latency (ms)
TLS 1.3	Server	No	No	1	0.21
OAuth 2.0 PKCE	Delegated	No	No	3+	N/A
FIDO2/WebAuthn	User	Partial	No	2	N/A
Standard ECDH	No	No	No	2	0.11
ECC-DTB-AKA	Yes	Yes	Yes	3	0.52

Table 5.2: Table 5.2: Authentication and key-exchange prior work vs. ECC-DTB-AKA.

5.2.6 EMV and Card-Present Authentication

EMV (Europay, Mastercard, Visa) chip cards use ECDSA on secp256r1 for offline and online transaction authentication. Contactless payments add relay-attack vulnerabilities addressed by distance-bounding protocols under research. EMV addresses card-present channels; ECC-DTB-AKA addresses remote digital banking

channels with similar ECC foundations but different binding requirements ($\text{device}_{\text{fp}}$, $\text{txn}_{\text{nonce}}$) [9][26][16].

5.3 Prior Work on Data Encryption, Security, and Integrity

This section reviews prior approaches to data encryption, security, and integrity in e-banking and e-finance server-server channels, distinct from the authentication focus of Section 5.2, and compares them with ECC-HISE [42][43].

5.3.1 TLS Record Layer Protection

TLS record layer encrypts application data using AEAD ciphers (AES-128-GCM or ChaCha20-Poly1305) with keys derived from the handshake. Protection ends at TLS termination: load balancers, API gateways, and service meshes decrypt and re-encrypt traffic at each hop. Stored or forwarded messages after termination lack persistent integrity proofs [2][1][28].

Figure 5.6: TLS record layer vs. persistent envelope integrity



Figure 5.6: TLS record layer vs. persistent envelope integrity (Figure 5.6).

5.3.2 ECIES and Hybrid Encryption

Elliptic Curve Integrated Encryption Scheme (ECIES, SEC 1 / IEEE 1363a) combines ECDH key agreement with symmetric encryption and MAC. Standard ECIES encrypts individual messages but does not provide hierarchical key management, batch Merkle audit, or domain-separated key derivation. ECC-HISE extends

ECIES concepts with batch-level Merkle roots and three-level HKDF hierarchy [3][5][6][41].

5.3.3 Static Pre-Shared Key File Encryption

Inter-bank file transfer historically uses PGP/GPG or proprietary tools with symmetric keys distributed annually via secure courier. Keys remain valid for extended periods; compromise exposes all files encrypted under that key. Benchmark: static AES-GCM PSK achieves 0.22 ms for 100 records but provides no signatures, Merkle audit, or key hierarchy [2][30].

5.3.4 Blockchain and Distributed Ledger Audit

Blockchain systems (Bitcoin, Ethereum, R3 Corda, Hyperledger Fabric) provide tamper-evident audit via Merkle trees and consensus. JPMorgan Onyx and similar platforms pilot distributed ledgers for settlement. Limitations: consensus latency, throughput constraints, and operational complexity. ECC-HISE adopts Merkle audit without distributed consensus, suitable for traditional settlement workflows [30][32][31].

Figure 5.7: Merkle audit — blockchain vs. ECC-HISE



Figure 5.7: Merkle audit: blockchain vs. ECC-HISE envelope (Figure 5.7).

5.3.5 Comparative Summary

Table 5.3 compares data-protection prior work with ECC-HISE for 100-record batches.

Approach	Hierarchy	Merkle audit	Per-record sig	Latency (ms)	Size (B)
TLS record only	No	No	No	0.24	28905
Static AES-GCM PSK	No	No	No	0.22	28400
Standard ECIES	No	No	Optional	N/A	N/A
Blockchain audit	Varies	Yes (consensus)	Varies	High	High
ECC-HISE	Yes	Yes	Yes	23.32	37823

Table 5.3: Table 5.3: Data encryption and integrity prior work vs. ECC-HISE.

5.3.6 Database TDE and Column-Level Encryption

Transparent Data Encryption (TDE) protects data at rest in Oracle, SQL Server, and PostgreSQL extensions. Column-level encryption targets sensitive fields (PAN, SSN). These mechanisms protect storage but not data in transit between services or audit integrity of exported settlement batches. ECC-HISE complements TDE by protecting inter-service batch transfer with verifiable integrity chains.

5.4 Synthesis: Vulnerabilities and Research Contributions

This section synthesizes findings from Sections 5.1 through 5.3, lists security vulnerabilities present in existing e-banking and e-finance systems, and identifies the issues addressed by this dissertation [42][43].

5.4.1 Vulnerabilities in Existing Systems

V1 — Decoupled authentication layers: TLS encrypts channels without binding user identity to session keys, enabling session hijacking after token theft [1][28].

V2 — Absence of device binding: Standard ECDH and TLS do not incorporate device fingerprints in key derivation [5][6][12][29].

V3 — Bearer token semantics: OAuth and JWT sessions are transferable to any client possessing the token [13][14].

V4 — Flat key hierarchies: Long-lived DEKs increase blast radius of key compromise.

V5 — Transport-only integrity: mTLS and TLS do not protect data after termination [1][28].

V6 — Audit friction: Manual reconciliation required without cryptographic batch proofs.

V7 — Static PSK rotation gaps: Infrequent key rotation in inter-bank file transfer.

V8 — Legacy TLS configurations: TLS 1.2 without forward secrecy in some deployments [1][8][28].

Vulnerability	Affected systems	Addressed by
V1 Decoupled auth	TLS + app login	ECC-DTB-AKA
V2 No device bind	TLS, ECDH	ECC-DTB-AKA
V3 Bearer tokens	OAuth, JWT	ECC-DTB-AKA (TBK HMAC)
V4 Flat keys	TDE, file PSK	ECC-HISE hierarchy
V5 Transport-only	mTLS, TLS	ECC-HISE envelope
V6 Audit friction	SIEM, manual	ECC-HISE Merkle
V7 Static PSK	PGP inter-bank	ECC-HISE per-batch SK
V8 Legacy TLS	Older gateways	ECC-DTB-AKA fixed alg

Table 5.4: Table 5.4: Vulnerabilities in existing e-banking systems and dissertation responses.

5.4.2 Research Contributions

This dissertation makes the following original contributions to information security in e-banking and e-finance systems: [42][43]

C1 — ECC-DTB-AKA: A novel three-round ECC-based authenticated key agreement protocol with device fingerprint and transaction nonce binding in HKDF derivation, formally analyzed and quantitatively evaluated [5][6][7][8].

C2 — ECC-HISE: A novel hierarchical secure envelope scheme with Merkle audit chains for server-server batch data transfer, providing confidentiality, integrity, and efficient audit verification [30].

C3 — Dual-layer methodology: An integrated architecture combining client-server and server-server ECC-based security in a coherent e-financial service design.

C4 — Prototype implementation: Working software demonstrating both schemes with REST APIs, benchmarks, and end-to-end integration tests.

C5 — Quantitative evaluation: Empirical comparison with TLS 1.3, ECDH, static PSK, and TLS record-layer baselines using reproducible benchmark suites

[1][12][29][28].

5.4.3 Limitations and Future Work

Limitations include: software-based key storage (no HSM in prototype); proof sketches rather than machine-verified proofs; simplified certificate handling; single-process settlement demo. Future work: post-quantum algorithm integration (ML-KEM, ML-DSA); HSM deployment; formal verification in ProVerif or Tamarin; large-scale field trial with partner institution; meta-Merkle trees for million-record national settlement [4][45][44][30].

5.4.4 Alignment with Regulatory Requirements

PCI DSS Requirement 4 mandates strong cryptography for transmission; Requirement 3 for storage. PSD2 Strong Customer Authentication aligns with ECC-DTB-AKA mutual authentication. SWIFT CSP controls for integrity and non-repudiation align with ECC-HISE signed Merkle batch headers. The proposed schemes support compliance objectives while exceeding minimum baselines through integrated device binding and cryptographic audit [17][18][20][24].

5.5 Chapter Summary

This chapter evaluated ECC-DTB-AKA and ECC-HISE against Chapter 1 attack mechanisms, reviewed prior work on authentication/key exchange (Section 5.2) and data encryption/integrity (Section 5.3), and synthesized eight identified vulnerabilities with five research contributions. Quantitative results confirm ECC-DTB-AKA achieves 0.52 ms authentication with superior security properties over TLS and ECDH, and ECC-HISE provides audit-capable batch protection at 23.32 ms per 100 records. The concluding remarks summarize contributions and outline future research directions [8][29][12][1].

Conclusions

5.6 Summary of Approach

This dissertation investigated information security in electronic banking and electronic finance service systems, with emphasis on elliptic curve cryptography for authentication, key exchange, and data protection. A systematic threat analysis (Chapter 1) established the attack landscape and identified critical limitations in deployed authentication mechanisms (Section 1.2) and data-security mechanisms (Section 1.3) [8][29][11][34].

Two novel ECC-based methodologies were proposed, analyzed, implemented, and evaluated: ECC-DTB-AKA for client-server authenticated key agreement with device and transaction binding (Chapter 2), and ECC-HISE for server-server hierarchical secure envelopes with Merkle audit chains (Chapter 3). Chapter 4 demonstrated integrated architecture and programmatic implementation. Chapter 5 evaluated effectiveness against network attacks, reviewed prior work, and synthesized vulnerability findings [7][8][30].

5.7 Contributions

The principal contributions of this research are:

First, the design and formal analysis of ECC-DTB-AKA, a three-round protocol achieving mutual authentication, forward secrecy, device binding, and per-transaction key derivation using SECP256R1 ECDH, ECDSA, and HKDF [1][8][5][6].

Second, the design and formal analysis of ECC-HISE, a hierarchical envelope scheme combining domain-separated key derivation, AES-256-GCM encryption, Ed25519 signatures, and Merkle-tree batch integrity for inter-bank settlement

[2][5][6][10].

Third, a dual-layer integrated security architecture unifying both schemes in an e-financial service system with defined trust boundaries and component interactions.

Fourth, a working software prototype with REST APIs, benchmark suites, and end-to-end integration tests providing reproducible quantitative evaluation.

Fifth, comprehensive prior-work analysis positioning the proposed schemes relative to TLS 1.3, OAuth 2.0 PKCE, FIDO2, ECIES, static PSK, and blockchain audit approaches [1][41][14][13].

5.8 Limitations

This research has limitations that should be acknowledged. The prototype uses software key storage rather than hardware security modules. Formal security proofs are presented as structured proof sketches rather than machine-verified derivations. Certificate management is simplified compared to production PKI. The settlement demonstration performs sender and receiver verification in a single process. These limitations do not invalidate the cryptographic designs but indicate areas requiring hardening for production deployment [4].

5.9 Future Work

Future research directions include: integration of NIST post-quantum algorithms (ML-KEM, ML-DSA) for long-term confidentiality; HSM-backed key storage and FIPS 140-3 validation; formal verification using ProVerif or Tamarin provers; field trials with partner financial institutions; meta-Merkle hierarchies for million-record national settlement; and composition with FIDO2/WebAuthn for phishing-resistant client key enrollment [45][44][15][30].

Appendix A

Appendix

A.1 A. Algorithm Listings

Algorithm A.1 (ECC-DTB-AKA Client Handshake): See Section 2.3.7. Algorithm A.2 (ECC-DTB-AKA Server Handshake): See Section 2.3.8. Algorithm A.3 (ECC-HISE Batch Envelope Construction): See Section 3.4.1–3.4.2. Algorithm A.4 (Merkle Root and Proof Verification): See `prototype/ecchise/merkle.py` [30].

A.2 B. API Specification

Prototype REST API (`prototype/api/app.py`): GET `/health` — returns status and supported schemes. POST `/auth/register` — body: `{deviceid}`; registers client key. POST `/auth/handshake/init` — body: `{deviceid}`; returns `sessionid` and handshake data. POST `/auth/handshake/complete` — body: `{sessionid, txnnonce_hex}`; completes authentication. POST `/settlement/submit-batch` — body: `{batchid, domainid, records[]}`; submits and verifies batch.

A.3 C. Benchmark Raw Data

Chapter 2 benchmarks: `thesis/benchmarks/chapter2benchmarks.json`. Chapter 3 benchmarks: `thesis/benchmarks/chapter3benchmarks.json`. Reproduce via: `python -m prototype.benchmarks.benchdtb_aka && python -m prototype.benchmarks.benchhise`.

A.4 D. Prototype Module Index

prototype/common/crypto_utils.py — SHA-256, HKDF, ECDH, ECDSA utilities.

prototype/ecc_dtb_aka/protocol.py — ECC-DTB-AKA client/server protocol. prototype/ecc_hise/hierar

— hierarchical key derivation. prototype/ecc_hise/merkle.py — Merkle tree construction and proofs. prototype/ecc_hise/envelope.py — secure batch envelope. prototype/api/app.py — FastAPI integration. prototype/benchmarks/ — timing and size measurement scripts [\[5\]](#)[\[6\]](#)[\[9\]](#)[\[26\]](#).

Appendix B

Scheme Design Document

Novel ECC-Based Security Schemes for E-Banking Systems

B.1 1. Overview

This document specifies two original cryptographic schemes proposed in this dissertation: (A) ECC-DTB-AKA — Device- and Transaction-Bound Authenticated Key Agreement for client-server e-banking communication, and (B) ECC-HISE — Hierarchical Integrity-preserving Secure Envelope for server-server inter-bank data exchange. Both schemes employ elliptic curve cryptography (ECC) as the foundational primitive, selected for its superior security-per-bit ratio compared to RSA and its widespread adoption in modern financial infrastructure standards including EMV, FIDO2, and TLS 1.3 [1][7][8][11].

The schemes are designed to address distinct vulnerability classes identified in Chapter 1: ECC-DTB-AKA targets authentication and session-key establishment weaknesses in client-server channels (Section 1.2), while ECC-HISE targets data confidentiality, integrity, and auditability weaknesses in server-server settlement and record-transfer channels (Section 1.3).

B.2 2. Scheme A: ECC-DTB-AKA

ECC-DTB-AKA establishes a mutually authenticated session key between a banking client C and banking server S , with the session key further bound to a device

fingerprint and a transaction nonce to enable per-transaction key derivation without additional round trips [5][6].

B.2.1 Notation

Let E be an elliptic curve over a finite field with base point G of prime order q . Let H denote SHA-256, and HKDF denote RFC 5869 key derivation. Denote scalar multiplication as $[k]G$. Entities: Client C with long-term key pair $(sk_C, PK_C = [sk_C]G)$; Server S with certificate chain rooted in bank CA ; $device_{fp} = H(hardware_{attributes})$ truncated to 128 bits; ephemeral pairs (ek_C, EK_C) and (ek_S, EK_S) generated per session [4][5][6][11].

B.2.2 Protocol Steps (3-Round Handshake)

Round 1 (Client Init): C generates ephemeral (ek_C, EK_C) , samples $nonce_c \lll PH3 \ggg \{0,1\}^{128}$, computes $t_C = H(EK_C \parallel PK_C \parallel device_{fp} \parallel nonce_c)$, and sends $MSG1 = (EK_C, PK_C, device_{fp}, nonce_c, Sig_C(sk_C, t_C))$.

Round 2 (Server Response): S verifies Sig_C and PK_C against customer registry. S generates (ek_S, EK_S) , samples $nonce_s$, computes $Z = [ek_S]EK_C = [ek_C]EK_S$, $MSK = HKDF(Z \parallel device_{fp} \parallel nonce_c \parallel nonce_s \parallel 'ECC-DTB-AKA-v1', 256)$, and sends $MSG2 = (EK_S, nonce_s, Cert_S, Sig_S(sk_S, H(EK_S \parallel nonce_s \parallel MSK)))$ [5][6].

Round 3 (Client Confirm): C verifies $Cert_S$ and Sig_S , recomputes MSK , derives $TBK_{txn} = HKDF(MSK \parallel txn_{nonce} \parallel 'TBK', 256)$ for a specific transaction, and sends $MSG3 = (HMAC(TBK_{txn}, txn_{nonce}), Sig_C(sk_C, H(MSK \parallel 'confirm')))$ [5][6].

Upon successful verification, both parties hold MSK (master session key) and can derive transaction-bound keys TBK_{txn} on demand.

B.2.3 Security Properties

Authenticity: Mutual authentication via ECDSA signatures and server certificate validation [4][9][26].

Forward secrecy: Compromise of long-term keys does not reveal past MSK values due to ECDH ephemerals [1][8][12][29].

Device binding: $device_{fp}$ included in HKDF salt prevents session transfer to unauthorized devices [5][6].

Transaction binding: TBK_{txn} incorporates $\text{txn}_{\text{nonce}}$, limiting replay to single transaction context.

Resistance: Replay (nonces + timestamps), MITM (signatures + cert chain), session hijacking ($\text{device}_{\text{fp}}$ binding).

B.3 3. Scheme B: ECC-HISE

ECC-HISE provides encrypted, signed, and auditable envelopes for batch financial records transferred between server nodes in an inter-bank or intra-bank settlement network.

B.3.1 Key Hierarchy

Level 0 — Master Key (MK): Stored in HSM at clearing-house authority; $MK \lll PH0 \ggg Z_q$.

Level 1 — Domain Keys: $DK_i = \text{HKDF}([MK]G \parallel \text{domain}_{\text{id}} \parallel \text{'DK'}, 256)$ for each business domain (retail, corporate, settlement) [5][6].

Level 2 — Session Keys: $SK_{\text{batch}} = \text{HKDF}(DK_i \parallel \text{batch}_{\text{id}} \parallel \text{epoch} \parallel \text{'SK'}, 256)$ rotated per batch [5][6].

B.3.2 Envelope Construction

For each record r_j in batch $B = \{r_1, \dots, r_n\}$: (1) Generate ephemeral eph_j ; compute shared Z_j via $\text{ECDH}(\text{eph}_j, PK_{\text{receiver}})$. (2) Derive $K_{\text{enc}} = \text{HKDF}(Z_j \parallel SK_{\text{batch}}, 256)$; split into K_{aes} (128-bit) and K_{mac} (128-bit). (3) $\text{ciphertext}_j = \text{AES-256-GCM}(K_{\text{aes}}, IV_j, r_j)$. (4) $\text{sig}_j = \text{Ed25519}(\text{sk}_{\text{sender}}, H(\text{ciphertext}_j \parallel \text{batch}_{\text{id}} \parallel j))$. (5) $\text{leaf}_j = H(\text{ciphertext}_j \parallel \text{sig}_j \parallel \text{metadata}_j)$. Merkle root $\text{Root} = \text{Merkle}(\text{leaf}_1, \dots, \text{leaf}_n)$ is included in batch header. Header = (batch_{id}, epoch, PK_{eph} , Root, Sig_{header}, timestamp) [2][5][6][10].

B.3.3 Verification and Audit

Receiver verifies Sig_{header}, reconstructs Merkle root from received records, decrypts each record, and validates sig_j. Any tampering breaks Merkle root or GCM authentication tag. Audit log stores (batch_{id}, Root, timestamp) immutably for regulatory compliance [2][30].

B.4 4. Comparison Baselines

ECC-DTB-AKA will be compared against: TLS 1.3 full handshake + application-layer OTP; standard ECDH without device/transaction binding; OAuth 2.0 PKCE flow [1][12][29][14].

ECC-HISE will be compared against: TLS record layer only; AES-GCM with static pre-shared keys; standard ECIES without hierarchical keys or Merkle audit chain [2][41][30][1].

Metrics: authentication latency (ms), ciphertext overhead (bytes), round-trip count, and qualitative attack-resistance matrix.

B.5 5. Implementation Mapping

Prototype modules: `prototype/ecc_dtb_aka/` (client, server, protocol), `prototype/ecc_hise/` (envelope, merkle, hierarchy), `prototype/benchmarks/` (timing and size measurements) [30].

Cryptographic library: Python cryptography (OpenSSL backend), curves: SECP256R1 and Ed25519 [10].

Bibliography

- [1] Rescorla, E.. “The Transport Layer Security (TLS) Protocol Version 1.3.” RFC 8446, IETF, 2018.
- [2] Dworkin, M.. “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM).” NIST SP 800-38D, 2007.
- [3] Barker, E.. “Recommendation for Key Management: Part 1 — General.” NIST SP 800-57 Part 1 Rev. 5, 2020.
- [4] Cooper, D. et al.. “Internet X.509 Public Key Infrastructure Certificate and CRL Profile.” RFC 5280, IETF, 2008.
- [5] Krawczyk, H.. “Cryptographic Extraction and Key Derivation: The HKDF Scheme.” CRYPTO 2010, LNCS, 2010.
- [6] Krawczyk, H.. “HMAC-based Extract-and-Expand Key Derivation Function (HKDF).” RFC 5869, IETF, 2010.
- [7] Bellare, M.; Rogaway, P.. “Entity Authentication and Key Distribution.” CRYPTO 1993, LNCS, 1993.
- [8] Canetti, R.; Krawczyk, H.. “Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels.” EUROCRYPT 2001, LNCS, 2001.
- [9] Johnson, D.; Menezes, A.; Vanstone, S.. “The Elliptic Curve Digital Signature Algorithm (ECDSA).” International Journal of Information Security, 2001.
- [10] Bernstein, D. J. et al.. “Edwards-Curve Digital Signature Algorithm (EdDSA).” RFC 8032, IETF, 2017.

- [11] SECG. “SEC 1: Elliptic Curve Cryptography, Version 2.0.” Standards for Efficient Cryptography Group, 2010.
- [12] ANSI. “Public Key Cryptography for the Financial Services Industry: Key Agreement Using Diffie-Hellman.” ANSI X9.63, 2011.
- [13] Hardt, D.. “The OAuth 2.0 Authorization Framework.” RFC 6749, IETF, 2012.
- [14] Lodderstedt, T. et al.. “OAuth 2.0 for Native Apps (PKCE).” RFC 8252, IETF, 2017.
- [15] Barth, A. et al.. “FIDO 2.0 Web Authentication (WebAuthn).” W3C Recommendation, 2021.
- [16] Brandt, A. et al.. “EMV Integrated Circuit Card Specifications for Payment Systems.” EMVCo, Book 2, 2023.
- [17] PCI Security Standards Council. “Payment Card Industry Data Security Standard (PCI DSS).” Version 4.0, 2022.
- [18] European Parliament and Council. “Directive (EU) 2015/2366 on Payment Services (PSD2).” Official Journal of the European Union, 2015.
- [19] ISO/IEC. “Information Security Management Systems — Requirements.” ISO/IEC 27001:2022, 2022.
- [20] SWIFT. “Customer Security Programme (CSP) — Security Controls.” SWIFT Handbook, 2024.
- [21] IBM Security. “Cost of a Data Breach Report 2024.” IBM Corporation, 2024.
- [22] Verizon. “Data Breach Investigations Report (DBIR) 2024.” Verizon Enterprise, 2024.
- [23] OWASP Foundation. “OWASP API Security Top 10 — 2023.” OWASP, 2023.
- [24] Bangladesh Bank / SWIFT. “Report on Cyber Heist Investigation (Bangladesh Bank).” Financial Sector Incident Analysis, 2016.
- [25] Kaspersky Lab. “The Great Bank Robbery: Carbanak Cybergang.” Securelist Research, 2015.

- [26] NIST. “Digital Signature Standard (DSS).” FIPS 186-5, 2023.
- [27] NIST. “Secure Hash Standard (SHS).” FIPS 180-4, 2015.
- [28] NIST. “Guidelines for the Selection, Configuration, and Use of TLS Implementations.” NIST SP 800-52 Rev. 2, 2019.
- [29] NIST. “Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography.” NIST SP 800-56A Rev. 3, 2018.
- [30] Merkle, R. C.. “A Digital Signature Based on a Conventional Encryption Function.” CRYPTO 1987, LNCS, 1988.
- [31] Nakamoto, S.. “Bitcoin: A Peer-to-Peer Electronic Cash System.” Bitcoin Whitepaper, 2008.
- [32] Cromwell, R. et al.. “Corda: An Introduction.” R3 Whitepaper, 2016.
- [33] Diffie, W.; Hellman, M.. “New Directions in Cryptography.” IEEE Transactions on Information Theory, 1976.
- [34] Miller, V. S.. “Use of Elliptic Curves in Cryptography.” CRYPTO 1985, LNCS, 1986.
- [35] Koblitz, N.. “Elliptic Curve Cryptosystems.” Mathematics of Computation, 1987.
- [36] Bellare, M.; Namprempre, C.. “Authenticated Encryption: Relations among Notions and Analysis of Generic Composition Paradigm.” ASIACRYPT 2000, LNCS, 2000.
- [37] Dolev, D.; Yao, A.. “On the Security of Public Key Protocols.” IEEE Transactions on Information Theory, 1983.
- [38] Open Banking UK. “Open Banking Standard — Security Profile.” Open Banking Implementation Entity, 2023.
- [39] NIST. “Post-Quantum Cryptography Standardization.” NIST CSRC — ML-KEM, ML-DSA, 2024.
- [40] Bernstein, D. J.. “Curve25519: New Diffie-Hellman Speed Records.” PKC 2006, LNCS, 2006.

- [41] IEEE. “IEEE Standard Specifications for Public-Key Cryptography — Amendment 1: ECIES.” IEEE 1363a, 2004.
- [42] Dennis, F. et al.. “A Comprehensive Survey on Electronic Banking Security.” *Computers & Security*, 2020.
- [43] Aloul, F. et al.. “Online Banking Security Measures and Vulnerabilities.” *IEEE Security & Privacy*, 2015.
- [44] Cremers, C. et al.. “The TAMARIN Prover for the Symbolic Analysis of Security Protocols.” *CAV 2013, LNCS*, 2013.
- [45] Blanchet, B.. “Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif.” *Foundations and Trends in Privacy and Security*, 2016.